ENASTIC
Ecole Nationale Supérieure des TIC

Projet de Fin d'Études

En vue de l'obtention du **Diplôme de Licence en Informatique**

THÈME

CyberSentinel-LLM :

Plateforme pour la Détection Contextuelle et la Neutralisation Adaptive des Menaces Réseau

Présenté par :

M. YAKHOUB MAHAMAT HAMDO

Encadrant :

Dr. MOUAZ MICKAIL

Enseignant chercheur en IA

Jury :

Président : M. _____

Rapporteur : M. _____

Année Universitaire : 2025–2026

AVANT-PROPOS

L'École Nationale Supérieure des Technologies de l'Information et de la Communication (ENASTIC) est un établissement public d'enseignement, de recherche, d'innovation et de formation supérieure dans les domaines des communications électroniques. Elle est créée par l'Ordonnance **N°005/PR/2015** du 02 mars 2015, dotée de la personnalité juridique et de l'autonomie financière.

Placée sous la tutelle du Ministère des Postes et des Nouvelles Technologies de l'Information et de la Communication, et du Ministère de l'Enseignement Supérieur, de la Recherche et de l'Innovation pour l'organisation académique et pédagogique, l'ENASTIC a pour missions de :

- ☞ Assurer la formation initiale et continue des cadres dans les domaines des Technologies de l'Information et de la Communication ;
- ☞ Entreprendre la recherche appliquée d'intérêt général, en vue de promouvoir l'innovation pour le développement socio-économique du pays ;
- ☞ Former des jeunes à la culture scientifique afin de les rendre aptes à élaborer et à porter des projets structurants adossés aux TIC.

Pour compléter leurs connaissances théoriques, les étudiants doivent préparer un Projet de Fin d'Études (PFE) afin de le présenter devant un jury de soutenance pour parachever leur cycle de Licence.

C'est dans cette optique que le présent mémoire vient marquer la fin de notre cursus en Informatique à l'ENASTIC, sur le thème : **« CYBERSENTINEL-LLM : PLATEFORME POUR LA DÉTECTION CONTEXTUELLE ET LA NEUTRALISATION ADAPTIVE DES MENACES RÉSEAU »**, dans le cadre de l'année universitaire 2025–2026.

Ce travail constitue une preuve de concept pédagogique combinant détection d'anomalies par Machine Learning, raisonnement contextuel par LLM enrichi de MITRE ATT&CK via RAG, et neutralisation adaptive des menaces par blocage applicatif à validation humaine, afin d'aider les analystes SOC à interpréter, prioriser et neutraliser les incidents de sécurité. Le travail porte sur la conception, l'implémentation d'un prototype fonctionnel, et l'évaluation de ses capacités dans un contexte local (Tchad).

DÉDICACE

Je dédie ce modeste travail à mes parents.

À mon défunt père, qui a su me soutenir jusqu'à ses derniers instants
et aussi à ma mère qui est notre pilier et qui demeure mon modèle.

tout ce que j'entreprends est pour eux.

REMERCIEMENTS

Le présent mémoire est le résultat d'un travail effectué avec l'aide et le soutien de nombreuses personnes, à qui j'adresse mes sincères remerciements.

Je tiens à exprimer ma profonde gratitude à :

- ☞ Ma famille et mes proches, pour leur soutien moral et leur encouragement constants tout au long de ce parcours ;
- ☞ L'équipe pédagogique de l'École Nationale Supérieure des Technologies de l'Information et de la Communication (ENASTIC), pour la qualité de la formation dispensée ;
- ☞ Mon Directeur de mémoire, pour la qualité de son encadrement, sa disponibilité et ses précieux conseils tout au long de la réalisation de ce projet.

CONTENTS

Avant-propos **I**

Dédicace **II**

Remerciements **III**

Liste des figures **V**

Liste des tableaux **VI**

Liste des Abréviations **VII**

Introduction Générale **1**

1 Généralités **4**
Introduction 4
1.1 Présentation des IDS et IPS 4
1.2 Limites des approches classiques 5
1.3 Apport du Machine Learning en détection d'anomalies 5
1.4 Apport des LLM et principe Human-in-the-Loop 6
1.5 RAG pour la cybersécurité 6
Conclusion 7

2 Analyse des Besoins et Spécifications **8**
Introduction 8
2.1 Cas d'usage principaux 8
2.2 Exigences fonctionnelles 9
2.3 Exigences non fonctionnelles 10
Conclusion 11

3 Conception Logicielle et Architecture **12**
Introduction 12
3.1 Architecture globale 12
3.2 Arborescence du projet 13
3.3 Description des composants backend 14
3.4 Description du frontend 15
3.5 Refactoring frontend — Architecture modulaire 16

3.6 Diagrammes UML . . . 17
3.6.1 Diagramme de classes . . . 17
3.6.2 Diagramme de séquence . . . 18
3.6.3 Diagramme d'activité . . . 19
Conclusion . . . 20

4 Implémentation **21**
Introduction . . . 21
4.1 Pipeline Machine Learning . . . 21
4.2 Module RAG — MITRE ATT&CK . . . 22
4.3 Module LLM et Score de Confiance . . . 23
4.4 Système de Neutralisation Adaptive . . . 25
4.5 Endpoints principaux de l'API . . . 26
4.6 SSE et communication temps réel . . . 27
4.7 Architecture frontend temps réel . . . 27
4.8 Interfaces utilisateur . . . 28
Conclusion . . . 33

5 Tests, Validation et Résultats **34**
Introduction . . . 34
5.1 Scénario de démonstration complet . . . 34
5.2 Mécanismes d'évaluation . . . 36
5.3 Résultats observés . . . 36
5.4 Tests automatisés . . . 37
Conclusion . . . 39

Conclusion Générale et Perspectives **40**

Bibliographie **42**

Annexes **43**
Annexe A : Architecture globale . . . 43
Annexe B : Exemples de payloads API . . . 44
Annexe C : Guide d'installation . . . 45

LIST OF FIGURES

2.1 Diagramme de cas d'utilisation de CyberSentinel-LLM 9

3.1 Architecture globale de CyberSentinel-LLM 13
3.2 Diagramme de classes de CyberSentinel-LLM 18
3.3 Diagramme de séquence — flux d'une alerte jusqu'à la neutralisation 19
3.4 Diagramme d'activité — traitement d'un log entrant 20

4.1 Tableau de bord SOC — KPIs temps réel, timeline 24h et carte GeoIP 29
4.2 Page Alertes — liste complète avec labellisation et gestion des blocages 29
4.3 Page Incidents — détail d'un incident avec explication LLM structurée 30
4.4 Page Chat SOC — console conversationnelle streaming avec historique 30
4.5 Page Paramètres — état des services, 2FA TOTP, gestion des utilisateurs et IPs bloquées 31
4.6 Page Modèle ML — métriques F1-score, précision, rappel et ROC-AUC 31
4.7 Page Rapports — statistiques par période, top attaquants et export CSV 32
4.8 Page Connexion — authentification et inscription avec gestion des sessions 32
4.9 Overlay mode vocal — orb animée, équaliseur et transcription en temps réel 33

5.1 Page Démo — pipeline IDS 5 étapes avec terminal live et carte GeoIP 35
5.2 Notification de blocage — countdown 30s, détails de l'attaque et validation humaine 35
5.3 Architecture globale de CyberSentinel-LLM 43

LIST OF TABLES

1.1 Comparaison IDS par signatures vs IDS comportemental 5
1.2 Comparaison Isolation Forest vs Local Outlier Factor 6

2.1 Exigences fonctionnelles de CyberSentinel-LLM 10

3.1 Description des composants backend 15

4.1 Features utilisées dans le pipeline ML 21
4.2 Échantillon de techniques MITRE ATT&CK indexées (697 au total dans ChromaDB) 23
4.3 Modes de défense du module de neutralisation 25
4.4 Endpoints principaux de l'API CyberSentinel-LLM 27

5.1 Métriques d'évaluation du module de détection 36
5.2 Synthèse des fonctionnalités — état final 37
5.3 Variables d'environnement principales 45

LISTE DES ABRÉVIATIONS

API : Application Programming Interface

ASN : Autonomous System Number

C2 : Command and Control

ChromaDB : Base de données vectorielle locale

DB : Database (Base de Données)

GeoIP : Geographic Internet Protocol

HTTP : HyperText Transfer Protocol

IA : Intelligence Artificielle

IDS : Intrusion Detection System

IF : Isolation Forest

IPS : Intrusion Prevention System

JSON : JavaScript Object Notation

JWT : JSON Web Token

LLM : Large Language Model

LOF : Local Outlier Factor

MITRE ATT&CK : Adversarial Tactics, Techniques and Common Knowledge

ML : Machine Learning

MVP : Minimum Viable Product

NER : Named Entity Recognition

ORM : Object Relational Mapping

PFE : Projet de Fin d'Études

RAG : Retrieval-Augmented Generation

RBAC : Role-Based Access Control

REST : Representational State Transfer

SIEM : Security Information and Event Management

SOC : Security Operations Center

SSH : Secure Shell

SSE : Server-Sent Events

STT : Speech-to-Text

TIC : Technologies de l'Information et de la Communication

TTS : Text-to-Speech

TTL : Time To Live

UI : User Interface

Vitest : Framework de tests JavaScript/TypeScript

INTRODUCTION GÉNÉRALE

Contexte et justification

Au Tchad, la digitalisation est en pleine croissance : en 2025, le pays compte **2,74 millions d'internautes** (13,2% de la population de 20,8 millions) et **14,5 millions de connexions mobiles actives** (69,8%). Des projets structurants sont en cours, tels que le Data Center National en voie de certification, des projets de fibre optique vers le Niger, la Libye et l'Égypte, et l'objectif ambitieux d'atteindre 30% de pénétration Internet d'ici 2030.

Cependant, cette digitalisation rapide s'accompagne de menaces croissantes. Le 10 janvier 2024, une attaque DDoS orchestrée par le groupe Anonymous Sudan a ciblé **Airtel Tchad**, le principal opérateur télécom du pays, causant une coupure internet nationale totale à impact massif. Les attaques SSH par force brute persistent régionalement sur les serveurs Linux, et les fraudes au Mobile Money explosent en Afrique centrale. Face à ces menaces, la maturité en cybersécurité du Tchad demeure limitée : peu d'outils SIEM ou SOC nationaux, dépendance à des audits étrangers, et un manque criant d'experts locaux malgré les efforts de l'ANSICE (30 cadres du secteur numérique formés en cybersécurité en 2025, lancement du Réseau national des RSSI en mai 2025, et audit de certification du Data Center National en février 2026).

Dans ce contexte, automatiser la détection, fournir une assistance explicative aux analystes SOC et neutraliser adaptativement les menaces constituent des besoins prioritaires. C'est pour répondre à ce défi que le projet **CyberSentinel-LLM** a été proposé : une plateforme cognitive qui combine détection d'anomalies par Machine Learning, raisonnement contextuel par LLM enrichi via RAG sur la base MITRE ATT&CK, et neutralisation adaptive par blocage applicatif à validation humaine.

Objectif général

Concevoir et réaliser une plateforme cognitive de cybersécurité conversationnelle, nommée **CyberSentinel-LLM**, qui combine détection d'anomalies (ML), raisonnement contextuel augmenté (RAG + MITRE ATT&CK), assistance conversationnelle par LLM local et neutralisation adaptive des menaces par

blocage applicatif à validation humaine, pour améliorer l'efficacité, l'explicabilité et la réactivité du triage d'alertes au sein d'un SOC.

Objectifs spécifiques

- ☞ Développer un pipeline d'ingestion de logs et de détection d'anomalies (IsolationForest / LOF + fallback heuristique) ;
- ☞ Implémenter un module RAG indexant 697 techniques MITRE ATT&CK dans ChromaDB pour enrichir le raisonnement du LLM ;
- ☞ Intégrer un LLM local (Ollama) pour fournir des explications structurées en JSON avec score de confiance pondéré ;
- ☞ Implémenter un système de neutralisation adaptive : blocage applicatif middleware avec liste noire mémoire, notification countdown et approbation humaine obligatoire (Human-in-the-Loop) ;
- ☞ Implémenter une API backend (FastAPI) et une interface frontend (Next.js) avec streaming temps réel (SSE) et mode voix (STT/TTS) ;
- ☞ Fournir des outils de labellisation humaine et des métriques d'évaluation (precision, recall, F1, roc_auc) ;
- ☞ Documenter et déployer un scénario de démonstration reproductible localement.

Question centrale

Comment concevoir une plateforme cyberdéfensive conversationnelle exploitant le raisonnement contextuel d'un LLM enrichi par RAG tout en maintenant une précision opérationnelle, une robustesse, une confiance mesurable et une capacité de neutralisation adaptive dans des environnements de menace dynamiques, avec une architecture simple et reproductible ?

Hypothèses

- ☞ **H1 :** Un pipeline minimaliste basé sur des features orientées SSH permet de démontrer un fil rouge pédagogique complet et cohérent ;
- ☞ **H2 :** Les algorithmes d'anomalie standards (IsolationForest, LOF) suffisent pour un MVP pédagogique combinés à un fallback heuristique robuste ;
- ☞ **H3 :** Un LLM local enrichi par RAG MITRE ATT&CK améliore significativement l'explicabilité et la précision des analyses d'incidents ;

☞ **H4 :** Un système de blocage applicatif avec validation humaine (Human-in-the-Loop) constitue une forme viable de neutralisation adaptive dans un contexte pédagogique.

Positionnement du projet

CyberSentinel-LLM se positionne comme une **plateforme cognitive de cybersécurité** (niveau Licence) : une démonstration technique et expérimentale destinée à valider la faisabilité d'un SOC augmenté par LLM avec capacité de neutralisation sur un périmètre restreint. Le projet ne prétend pas à une solution industrielle, mais pose les bases conceptuelles et techniques d'une telle approche dans le contexte tchadien.

CHAPTER 1

GÉNÉRALITÉS

Introduction

La cybersécurité est devenue un enjeu stratégique majeur pour les organisations à l'échelle mondiale. Face à la sophistication croissante des attaques informatiques, les systèmes de détection d'intrusions (IDS) et de prévention (IPS) constituent la première ligne de défense des infrastructures numériques. Ce chapitre présente sur ces systèmes, leurs limites, et l'apport combiné du Machine Learning, des modèles de langage et des techniques RAG pour les renforcer.

1.1 Présentation des IDS et IPS

Un **système de détection d'intrusions (IDS)** est un outil de surveillance qui analyse le trafic réseau ou les journaux système afin d'identifier des activités suspectes ou malveillantes. Un **système de prévention d'intrusions (IPS)** va plus loin en bloquant automatiquement les menaces détectées. On distingue deux grandes approches :

- ☞ **Approches par signatures :** comparaison du trafic observé avec une base de signatures d'attaques connues. Outils phares : *Snort* et *Suricata*. Très efficaces contre les attaques connues, mais incapables de détecter les menaces inconnues (zero-day) ;
- ☞ **Approches comportementales / anomalies :** établissement d'un profil de comportement normal du système, toute déviation significative constituant une alerte. Plus adaptées à la détection de nouvelles menaces, mais sujettes à un taux de faux positifs élevé.

Table 1.1: Comparaison IDS par signatures vs IDS comportemental

Critère	**Par signatures**	**Comportemental**
Détection menaces connues	Excellente	Moyenne
Détection zero-day	Nulle	Bonne
Taux de faux positifs	Faible	Élevé
Mise à jour requise	Fréquente (base signatures)	Non
Exemples d'outils	Snort, Suricata	IsolationForest, LOF

1.2 Limites des approches classiques

Malgré leur efficacité sur les menaces connues, les IDS/IPS classiques présentent plusieurs limites importantes dans un contexte opérationnel moderne :

- ☞ **Volume d'alertes saturant :** les SOC reçoivent des milliers d'alertes quotidiennes, rendant le triage manuel pratiquement impossible sans outillage ;
- ☞ **Taux élevé de faux positifs :** une grande proportion des alertes ne correspond pas à de vraies attaques, épuisant les analystes et masquant les menaces réelles ;
- ☞ **Manque d'explicabilité :** les outils classiques signalent une anomalie sans en expliquer le contexte ni proposer de piste d'investigation ;
- ☞ **Fragmentation contextuelle :** les alertes sont souvent isolées, sans corrélation automatique avec d'autres événements permettant de reconstituer un scénario d'attaque complet.

1.3 Apport du Machine Learning en détection d'anomalies

Le Machine Learning qui par définition est une branche de l'Intelligence Artificielle permettant d'apprendre à partir de données, apporte une réponse aux limites des approches classiques en permettant d'apprendre automatiquement le comportement normal d'un système et de détecter les déviations significatives. Deux algorithmes non supervisés sont particulièrement pertinents pour la détection d'anomalies :

- ☞ **Isolation Forest (IF) :** algorithme basé sur l'isolation aléatoire des points. Les anomalies étant rares et distinctes, elles sont isolées plus rapidement que les points normaux. Le score d'anomalie est transformé via une fonction sigmoïde pour obtenir une valeur normalisée entre 0 et 1 [1] ;
- ☞ **Local Outlier Factor (LOF) :** algorithme basé sur la densité locale. Un point est considéré comme une anomalie si sa densité locale est significativement inférieure à celle de ses voisins [2].

Ces approches présentent néanmoins des limites : sensibilité au choix des features, fragilité face aux changements de domaine (*domain-shift*), et nécessité d'un volume minimal de données d'entraînement.

Table 1.2: Comparaison Isolation Forest vs Local Outlier Factor

Critère	**Isolation Forest**	**Local Outlier Factor**
Type	Ensemble (arbres)	Densité locale
Complexité	Faible	Moyenne
Données requises	Peu	Moyenne
Interprétabilité	Faible	Faible
Sensibilité au bruit	Robuste	Sensible
Usage dans le projet	Score principal	Score secondaire

1.4 Apport des LLM et principe Human-in-the-Loop

Les **Grands Modèles de Langage (LLM)** ouvrent de nouvelles perspectives pour l'assistance aux analystes SOC [4]. Contrairement aux IDS classiques, un LLM est capable de :

- ☞ **Reformuler et contextualiser** une alerte technique en langage naturel compréhensible ;
- ☞ **Synthétiser** plusieurs événements corrélés pour reconstituer un scénario d'attaque plausible ;
- ☞ **Proposer des hypothèses et des actions** d'investigation non-exécutives, laissant la décision finale à l'analyste humain [5].

Le principe **Human-in-the-Loop** est fondamental dans ce contexte : le LLM n'agit jamais de façon autonome sur le système, mais guide et assiste l'analyste dans sa prise de décision. Cela limite les risques d'actions automatiques erronées tout en bénéficiant de la capacité de raisonnement contextuel du LLM.

Dans le cadre de **CyberSentinel-LLM**, le modèle *qwen2.5:1.5b* via **Ollama** est utilisé localement, garantissant la confidentialité des données et l'indépendance vis-à-vis de services cloud externes.

1.5 RAG pour la cybersécurité

La technique **RAG (Retrieval-Augmented Generation)**, introduite par Lewis et al. [3], permet d'enrichir les réponses d'un LLM en lui fournissant un contexte externe pertinent récupéré dynamiquement depuis une base de connaissances.

Dans le contexte cybersécurité, le RAG présente plusieurs avantages majeurs :

- ☞ **Ancrage factuel :** le LLM ne génère plus de réponses basées uniquement sur son entraînement général, mais sur des références cybersécurité précises (MITRE ATT&CK, CVE) ;

- ☞ **Mise à jour sans réentraînement :** la base de connaissances peut être enrichie de nouvelles techniques d'attaque sans modifier le modèle ;
- ☞ **Traçabilité :** chaque explication peut être référencée à une technique MITRE précise, renforçant la confiance dans l'analyse.

Le framework **MITRE ATT&CK** (version 18, Enterprise) recense actuellement **216 techniques et 475 sous-techniques** [7]. Il constitue la base de connaissances idéale pour ce RAG : il catalogue les tactiques, techniques et procédures (TTP) utilisées par les acteurs malveillants, avec pour chaque technique une description, des indicateurs de détection et des mesures d'atténuation.

Dans **CyberSentinel-LLM**, **697 techniques MITRE ATT&CK** (version Enterprise, parsées depuis le bundle STIX officiel via `scripts/fetch_mitre.py`) sont indexées dans ChromaDB avec le modèle d'embedding *all-MiniLM-L6-v2* (384 dimensions, CPU-only), permettant une recherche sémantique locale sans dépendance cloud.

Conclusion

Ce chapitre a présenté l'évolution des systèmes de détection d'intrusions, leurs limites, et l'apport combiné du Machine Learning, des LLM et du RAG pour renforcer les capacités des SOC. Ces éléments justifient l'approche adoptée dans **CyberSentinel-LLM** : pipeline ML pour la détection, RAG MITRE ATT&CK pour le raisonnement contextuel, et blocage applicatif à validation humaine pour la neutralisation. Le chapitre suivant analysera les besoins fonctionnels et non fonctionnels du système.

CHAPTER 2

ANALYSE DES BESOINS ET SPÉCIFICATIONS

Introduction

Ce chapitre présente l'analyse des besoins qui ont guidé la conception de **CyberSentinel-LLM**. Compte tenu du contexte tchadien (faibles ressources en cybersécurité, absence de SOC nationaux structurés, et dépendance à des solutions étrangères), la plateforme doit répondre à des exigences précises tout en restant déployable localement sur une machine modeste.

2.1 Cas d'usage principaux

Les cas d'usage de **CyberSentinel-LLM** ont été définis à partir des besoins opérationnels d'un analyste SOC confronté à des attaques réseau :

- ☞ **Détection automatique d'anomalies :** ingestion de logs réseau (SSH, authentification), scoring ML et génération automatique d'alertes avec niveau de sévérité ;
- ☞ **Triage et gestion des incidents :** regroupement d'alertes corrélées en incidents, suivi du statut et de la confiance ;
- ☞ **Explication d'incidents via LLM + RAG :** génération automatique d'une analyse contextuelle en langage naturel enrichie de techniques MITRE ATT&CK pour chaque incident ;
- ☞ **Neutralisation adaptive :** notification urgente avec countdown, approbation humaine du blocage applicatif et génération des commandes iptables correspondantes ;
- ☞ **Tableau de bord SOC :** visualisation des KPI, timeline des alertes et cartographie GeoIP des sources d'attaque ;
- ☞ **Interface conversationnelle :** chat en langage naturel avec le LLM en mode texte et voix (STT/TTS locaux) ;

☞ **Labellisation et métriques de calibration :** validation humaine des alertes (`true_attack` / `false_positive`) et calcul de métriques (precision, recall, F1, roc_auc).

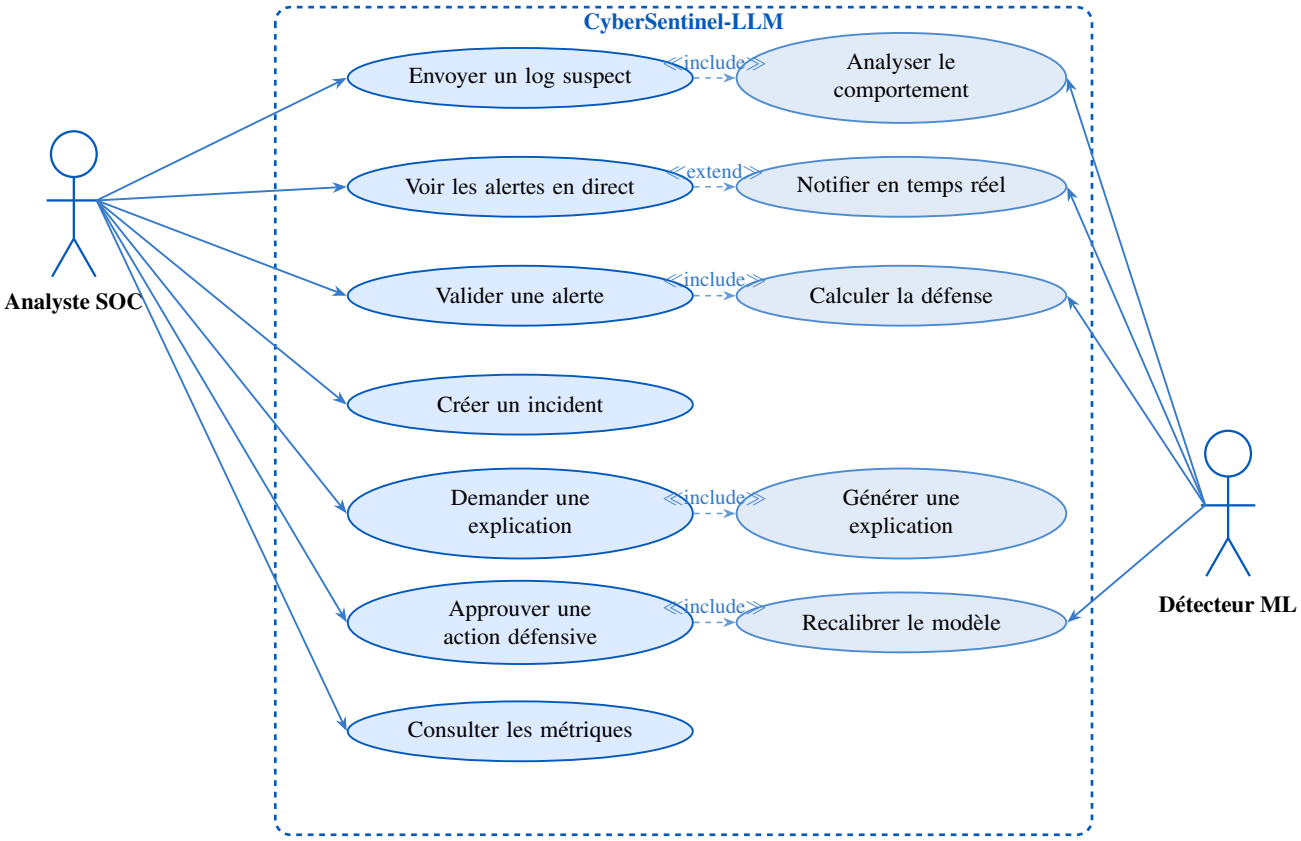


Figure 2.1: Diagramme de cas d'utilisation de CyberSentinel-LLM

2.2 Exigences fonctionnelles

Les exigences fonctionnelles définissent les fonctionnalités que le système doit obligatoirement implémenter. Les treize exigences identifiées (EF01 à EF13), présentées dans le Tableau 2.1, couvrent l'ensemble du cycle opérationnel de la plateforme : ingestion et scoring des logs, gestion des alertes et des incidents, recherche contextuelle par RAG MITRE ATT&CK, explication structurée par LLM, neutralisation adaptive des menaces, reporting et authentification à double facteur.

Table 2.1: Exigences fonctionnelles de CyberSentinel-LLM

ID	Fonctionnalité	Description
EF01	Ingestion de logs	Recevoir et traiter 1..N logs via POST /ingest/logs
EF02	Scoring ML	Calculer un score d'anomalie 0..1 par log ingéré (IF/LOF/hybride)
EF03	Gestion des alertes	Lister, filtrer et labelliser les alertes générées
EF04	Gestion des incidents	Créer, consulter et suivre les incidents
EF05	RAG MITRE ATT&CK	Indexer 697 techniques et enrichir le contexte LLM
EF06	Explication LLM	Générer une explication JSON structurée d'un incident
EF07	Score de confiance	Calculer confidence = 0.4×ML + 0.3×RAG + 0.3×LLM
EF08	Neutralisation adaptive	Blocage applicatif middleware avec liste noire mémoire
EF09	Notification blocage	Afficher countdown 30s et demander approbation humaine
EF10	Chat conversationnel	Interface de chat avec le LLM local (texte + voix)
EF11	Streaming temps réel	Diffuser les alertes en temps réel via SSE
EF12	Cartographie GeoIP	Afficher la localisation géographique des IPs sources
EF13	Métriques ML	Calculer et afficher precision/recall/F1/roc_auc

2.3 Exigences non fonctionnelles

Les exigences non fonctionnelles définissent les contraintes de qualité, de performance et de sécurité du système :

- ☞ **Réplicabilité locale :** le système doit être entièrement déployable sur une machine locale sans dépendance à des services cloud (Ollama local, SQLite, ChromaDB local) ;
- ☞ **Compatibilité matérielle contrainte :** toute la chaîne de traitement (ML + embedding RAG + LLM) doit fonctionner sur CPU Intel i7 11ème génération avec 16 GB de RAM, sans GPU ;
- ☞ **Performance minimale :** le scoring ML doit s'exécuter en moins de 50 ms par alerte ; l'explication LLM est tolérée entre 30 et 90 secondes sur CPU ;
- ☞ **Sécurité minimale :** CORS configurable, rate limiting par IP, security headers HTTP, limitation de taille des requêtes ;
- ☞ **Maintenabilité :** architecture modulaire (routers / services / models) facilitant l'extension et la maintenance ;

☞ **Principe Human-in-the-Loop :** le LLM ne prend aucune décision autonome sur le système ; toute action défensive de type blocage requiert une approbation humaine explicite.

Conclusion

Ce chapitre a défini les cas d'usage, les exigences fonctionnelles et non fonctionnelles de **CyberSentinel-LLM**. Ces spécifications guideront la conception architecturale et l'implémentation détaillée présentées dans les chapitres suivants.

CHAPTER 3

CONCEPTION LOGICIELLE ET ARCHITECTURE

Introduction

Ce chapitre présente l'architecture globale de **CyberSentinel-LLM**, la structure du code, la description détaillée des composants backend et frontend, ainsi que les diagrammes UML illustrant les flux et les interactions entre les différents modules du système.

3.1 Architecture globale

CyberSentinel-LLM repose sur une **architecture monolithique modulaire orientée API en cinq couches**. Ce choix architectural garantit la simplicité du déploiement local tout en permettant une séparation claire des responsabilités entre les composantes du système.

L'architecture comprend cinq grandes couches :

- **Couche 1 — Frontend (UI) :** interface Next.js 16 (App Router), 9 pages SOC (dashboard, alertes, incidents, demo, chat, modèle ML, paramètres, rapports, login), intégration SSE temps réel ;
- **Couche 2 — API Gateway :** FastAPI modulaire avec routers, endpoints, middlewares de sécurité et middleware de blocage IP ;
- **Couche 3 — Détection ML :** pipeline non supervisé + heuristique (IsolationForest, LOF, fallback), scoring 0..1, règles de sévérité ;
- **Couche 4 — RAG + LLM :** ChromaDB avec 697 techniques MITRE ATT&CK (bundle STIX Enterprise), embedding all-MiniLM-L6-v2, Ollama pour explication structurée et chat conversationnel ;
- **Couche 5 — Défense :** moteur de décision défensive, liste noire mémoire O(1), middleware blocage IP (HTTP 403), génération commandes iptables.

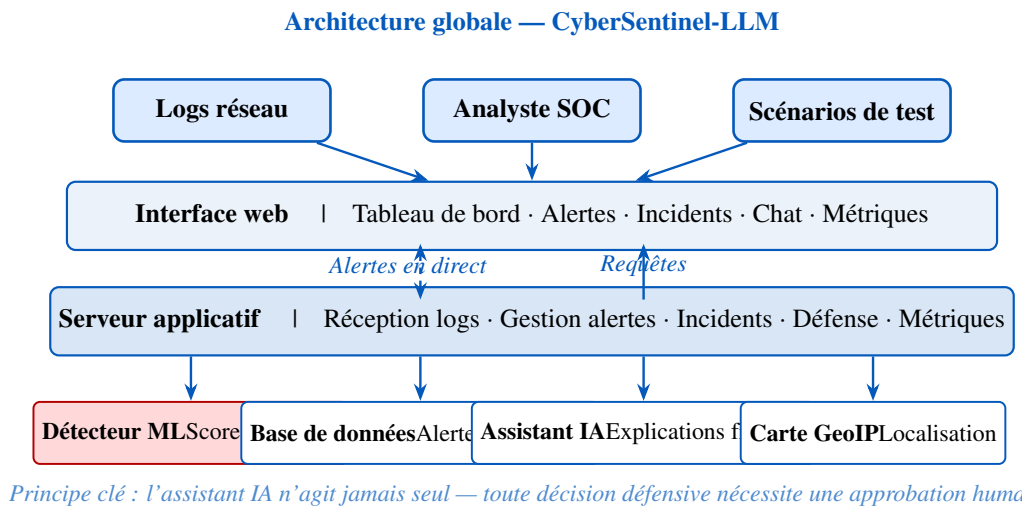


Figure 3.1: Architecture globale de CyberSentinel-LLM

3.2 Arborescence du projet

Le projet est organisé selon la structure suivante :

Structure du projet CyberSentinel-LLM

```
cybersentinel-llm/
|-- backend/
|   |-- app/
|   |   |-- core/           # Configuration et settings
|   |   |-- db/             # Connexion base de donnees
|   |   |-- routers/        # Contrats API metier
|   |   |-- endpoints/      # Endpoints LLM avances (chat, TTS, STT)
|   |   |-- models/         # Entites SQLModel (8 tables)
|   |   `-- services/       # Logique technique transverse
|   |       |-- detection.py    # ML : IF, LOF, heuristique
|   |       |-- rag_service.py  # ChromaDB + MITRE ATT&CK
|   |       |-- llm_ollama.py   # Ollama + RAG + confiance
|   |       |-- ip_blacklist.py # Liste noire memoire
|   |       |-- defense.py      # Moteur decision defensive
|   |       |-- geoip.py        # Lookup MaxMind GeoLite2
|   |       |-- sse.py          # Server-Sent Events
|   |       |-- tts_service.py  # Synthese vocale Piper
|   |       `-- whisper_service.py  # Transcription Whisper
|-- frontend/
|   |-- app/
|   |   |-- dashboard/      # Vue temps reel
|   |   |-- alerts/         # Gestion alertes + blocage
|   |   |-- incidents/      # Incidents + explication LLM
|   |   |-- demo/           # Pipeline 5 etapes anime
|   |   |-- chat/           # Chat SOC texte + voix
|   |   |-- model/          # Metriques ML
|   |   |-- settings/       # Parametres + IPs bloquees
|   |   |-- reports/        # Rapports + export CSV
|   |   `-- login/          # Authentification
|   `-- components/
|       |-- chat/
|       |   |-- types.ts             # Types partages (Conversation, ChatMsg)
|       |   |-- ConversationSidebar.tsx  # Sidebar conversations
|       |   |-- MessageRenderer.tsx  # Rendu messages + helpers
|       |   |-- OverviewCards.tsx    # Cartes alertes ML
|       |   `-- VoiceModeOverlay.tsx # Overlay mode vocal
|       |-- settings/
|       |   `-- shared.tsx           # StatusChip, ServiceRow, CopyBtn...
|       |-- BlockingNotification.tsx  # Notification blocage
|       |-- WorldMap.tsx              # Carte GeoIP Leaflet
|       `-- ...
`-- chroma_db/              # Persistance ChromaDB locale
```

3.3 Description des composants backend

Le backend est organisé en modules distincts, chacun ayant une responsabilité clairement définie :

Table 3.1: Description des composants backend

Composant	Rôle
`services/detection.py`	Extraction de 5 features, entraînement IF/LOF, scoring 0..1, fallback heuristique
`services/rag_service.py`	ChromaDB, embedding all-MiniLM-L6-v2, 697 techniques MITRE ATT&CK (STIX Enterprise), retrieve_context()
`services/llm_ollama.py`	Prompt RAG enrichi, appel Ollama, parsing JSON, retry, confiance pondérée
`services/ip_blacklist.py`	Liste noire mémoire thread-safe, synchronisation DB, boucle d'expiration
`services/defense.py`	Moteur de décision défensive (4 modes), TTL blocage
`services/geoip.py`	Lookup MaxMind GeoLite2-City.mmdb
`services/sse.py`	Queues asyncio multi-abonnés, publication alertes temps réel
`services/tts_service.py`	Synthèse vocale Piper locale
`services/whisper_service.py`	Transcription vocale STT locale
`routers/ingest.py`	Ingestion logs, scoring ML, GeoIP, SSE
`routers/defense.py`	Endpoints approve/revoke/block-manual
`routers/metrics.py`	Calcul precision/recall/F1/roc_auc
`models/`	Alert, Incident, LLMTrace, IPEnrichment, AlertLabel, Conversation, ChatMessage, DefenseAction

3.4 Description du frontend

Le frontend est développé avec **Next.js 16** (App Router) en mode dark cybersécurité et offre une interface SOC complète en 9 pages :

- ☞ **/dashboard :** vue temps réel avec 4 KPIs, timeline 24h, distribution sévérité, carte GeoIP, 10 dernières alertes, badge LIVE ;
- ☞ **/alerts :** liste complète des alertes, labellisation (true_attack / false_positive), approbation/révocation des blocages ;
- ☞ **/incidents/[id] :** détail d'un incident, explication LLM enrichie RAG, techniques MITRE identifiées, décision humaine ;
- ☞ **/demo :** pipeline IDS 5 étapes animé avec terminal live, carte GeoIP, explication LLM et blocage automatisé ;
- ☞ **/chat :** console conversationnelle streaming avec historique persisté, mode voix (STT Whisper + TTS Piper) ;

- ☞ **/model :** métriques ML (F1-score, précision, rappel, ROC-AUC, seuil actif) ;
- ☞ **/settings :** état des services (Ollama, SQLite, RAG, GeoLite2), configuration du mode défense, gestion des IPs bloquées (liste, blocage manuel, commandes iptables), authentification 2FA TOTP (QR code, activation/désactivation), gestion des utilisateurs (CRUD) et administration système (réinitialisation de la base de données) ;
- ☞ **/reports :** statistiques par période, top attaquants, distribution MITRE, export CSV/TXT ;
- ☞ **/login :** authentification et inscription.

3.5 Refactoring frontend — Architecture modulaire

Afin d'améliorer la maintenabilité et la lisibilité du code frontend, une refactorisation modulaire a été appliquée aux pages principales. L'objectif est de réduire la taille des fichiers monolithiques (plus de 1000 lignes) en extrayant des sous-composants réutilisables et des types partagés.

Refactoring de la page Chat

- ☞ **Fichier créé :** `components/chat/types.ts` — Types partagés (`Conversation`, `ChatMsg`, `OVERVIEW_MARKER`) ;
- ☞ **Composant OverviewCards :** affichage des cartes alertes ML détectées dans les réponses LLM ;
- ☞ **Composant ConversationSidebar :** sidebar de gestion des conversations (créer, renommer, supprimer) ;
- ☞ **Composant MessageRenderer :** rendu des messages avec helpers (Linkified, InlineCode, CodeBlock) et support TTS ;
- ☞ **Composant VoiceModeOverlay :** overlay plein écran pour le mode vocal avec animations CSS (orb breathing, ripple rings, equalizer).

Résultat : réduction de `chat/page.tsx` de 1395 à 884 lignes (-37%), avec une séparation claire des responsabilités.

Refactoring de la page Settings

- ☞ **Fichier créé :** `components/settings/shared.tsx` — Composants partagés (`StatusChip`, `ServiceRow`, `CopyBtn`, `ExpiryCountdown`) et utilitaires (`timeLeft`) ;
- ☞ **Composants extraits :**

- `ServiceStatusSection` — État des services (Ollama, SQLite, RAG, GeoLite2) ;
- `DefenseConfigSection` — Configuration défense + mode actif ;
- `BlockedIpsSection` — Liste IPs bloquées + blocage manuel ;
- `MfaSection` — Configuration 2FA (QR code, validation) ;
- `UsersSection` — Gestion utilisateurs (CRUD) ;
- `SystemAdminSection` — Administration système (reset DB).

Nettoyage du code mort

☞ **Fichier supprimé :** `lib/llm.ts` — Non utilisé (remplacé par appel direct Ollama) ;

☞ **Fonctions supprimées dans utils.ts :** `formatTimestamp()`, `severityColor()`, `severityIcon(` — Jamais utilisées hors fichier ;

☞ **Consolidation localStorage :** constantes dupliquées dans `lib/actor.ts` remplacées par imports depuis `AuthProvider`.

3.6 Diagrammes UML

Cette section présente les principaux diagrammes UML modélisant la structure statique et les flux dynamiques de **CyberSentinel-LLM** : le diagramme de classes décrivant le schéma de données, le diagramme de séquence illustrant le traitement d'une alerte de bout en bout, et le diagramme d'activité représentant la logique de décision du pipeline de détection.

3.6.1 Diagramme de classes

La Figure 3.2 présente les entités persistées en base de données et leurs relations. L'entité `Alert` centralise les événements détectés et peut être regroupée en `Incident`, chacun disposant d'une trace d'explication LLM (`LLMTrace`). Les actions défensives (`DefenseAction`) et les labels de validation humaine (`AlertLabel`) sont rattachés aux alertes, tandis que les conversations du chat SOC (`Conversation`, `ChatMessage`) forment un sous-graphe indépendant.

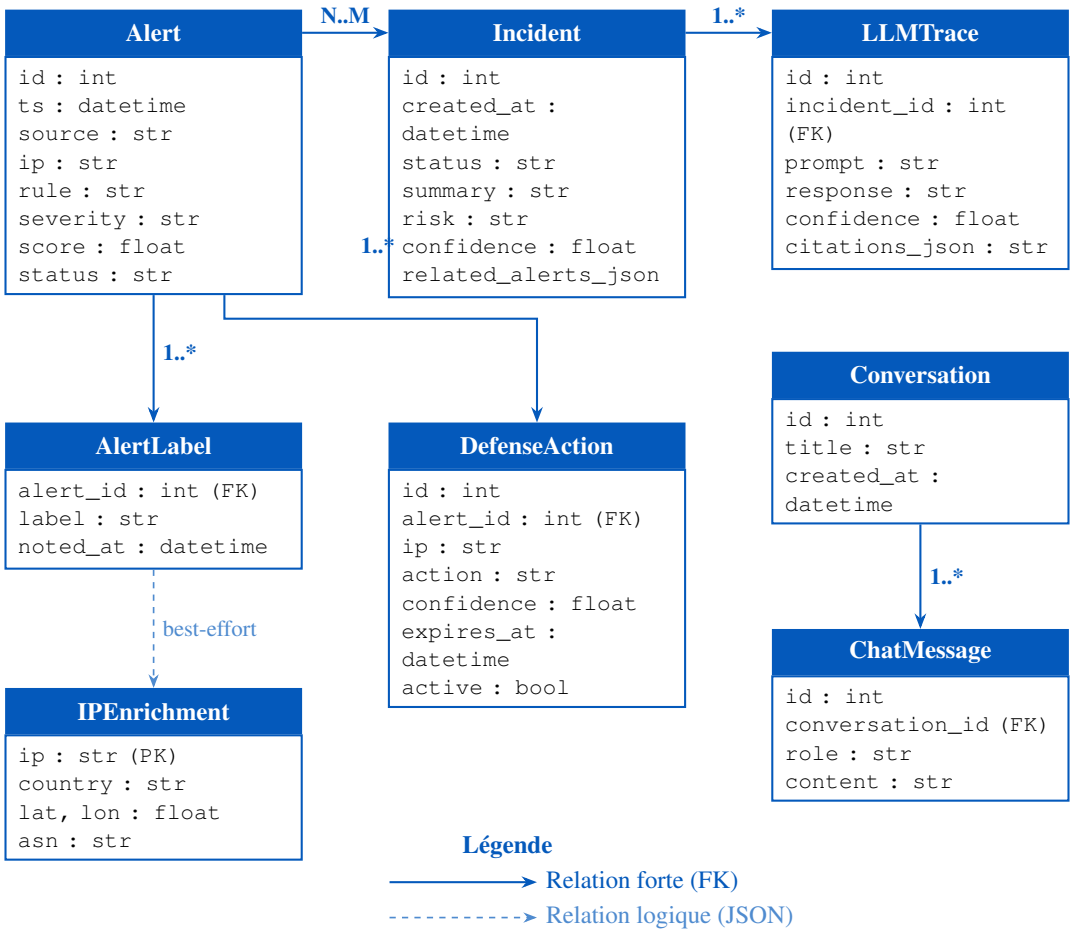


Figure 3.2: Diagramme de classes de CyberSentinel-LLM

3.6.2 Diagramme de séquence

La Figure 3.3 illustre le flux complet d'une alerte depuis l'arrivée d'un log suspect jusqu'à l'affichage de l'explication structurée à l'analyste SOC. Le log est d'abord soumis au pipeline ML qui calcule un score d'anomalie et enregistre l'alerte en base. Le moteur de défense détermine alors l'action appropriée (monitor, review ou block_temp) et déclenche une notification temps réel via SSE. L'analyste consulte l'alerte, approuve ou rejette l'action défensive, ce qui provoque la création d'un incident et l'appel au module RAG + LLM pour générer l'explication contextuelle finale.

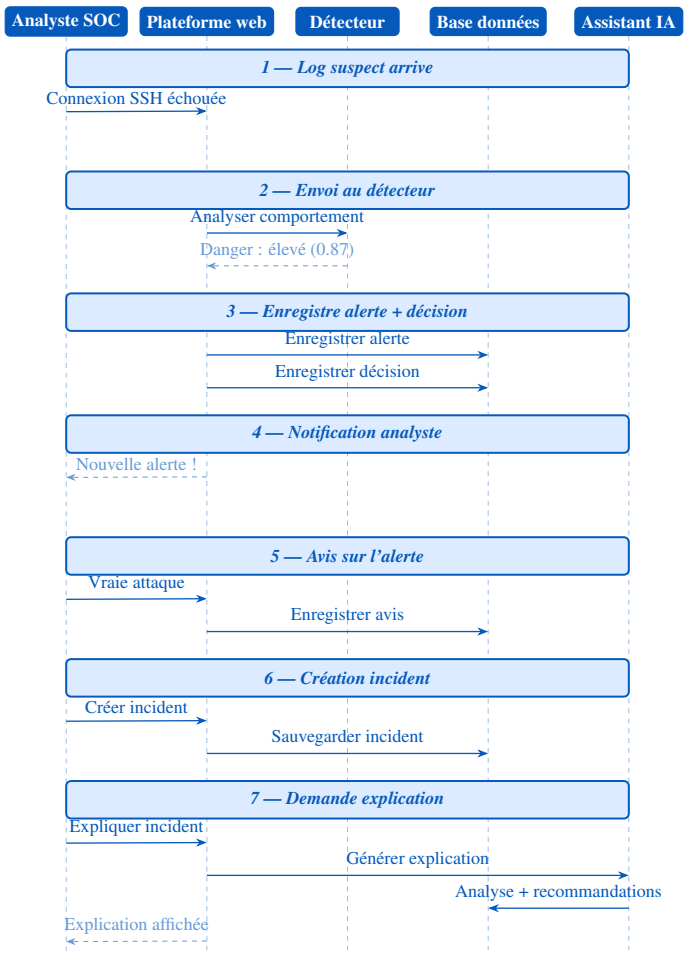


Figure 3.3: Diagramme de séquence — flux d'une alerte jusqu'à la neutralisation

3.6.3 Diagramme d'activité

La Figure 3.4 décrit le processus de traitement d'un log entrant sous forme d'activités séquentielles et de branchements conditionnels. Après l'analyse comportementale, le score d'anomalie oriente le flux : un score faible conduit à l'archivage du log sans alerte, tandis qu'un score élevé déclenche la classification à haute sévérité, l'enregistrement de l'alerte et la notification de l'analyste. L'analyste valide ou rejette l'alerte, ce qui détermine la création ou non d'un incident ; dans les deux cas, son retour (`true_attack` / `false_positive`) est utilisé pour calibrer le détecteur ML.

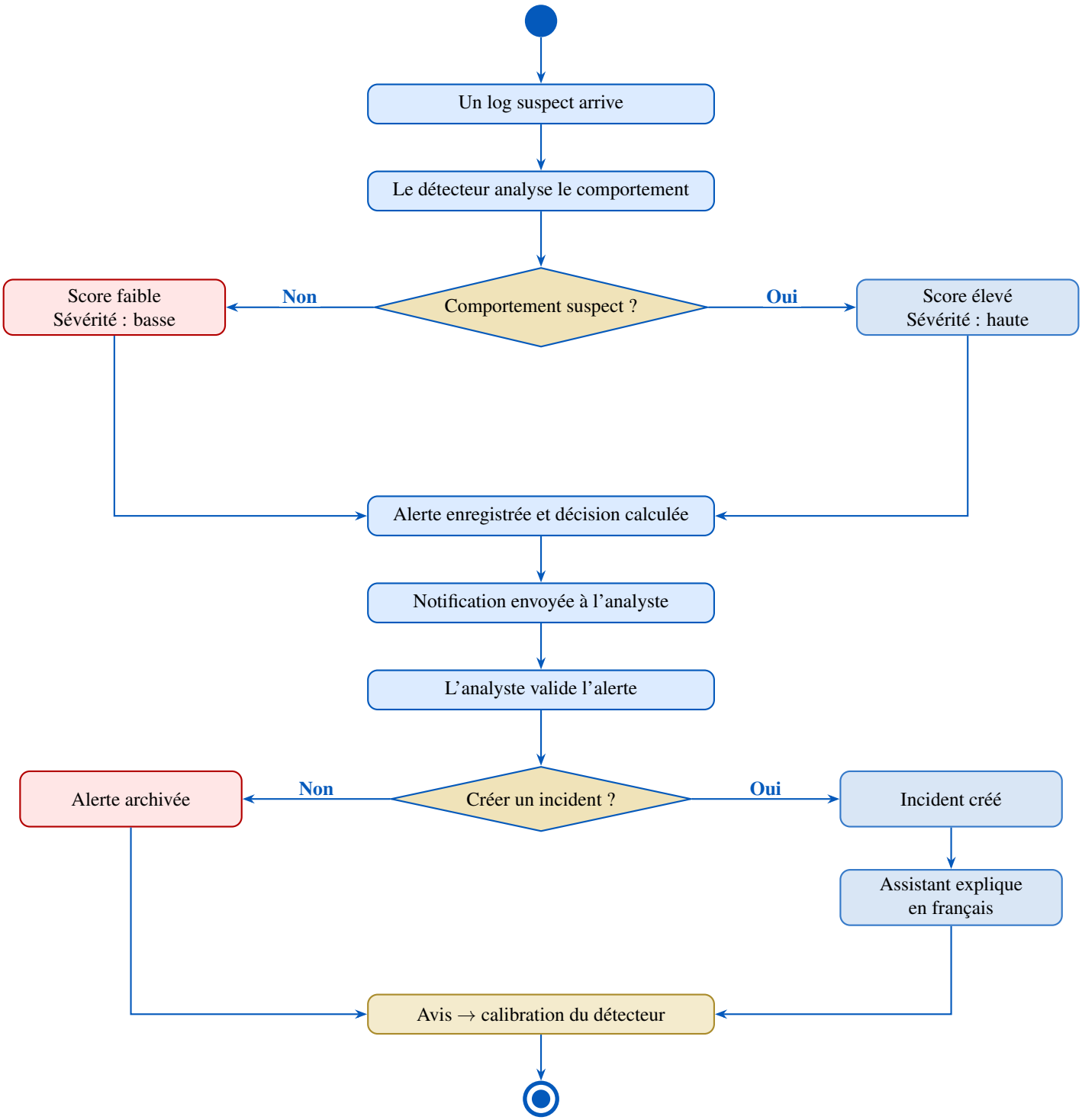


Figure 3.4: Diagramme d'activité — traitement d'un log entrant

Conclusion

Ce chapitre a présenté l'architecture globale en cinq couches, la structure du projet et les composants principaux de **CyberSentinel-LLM**. Les diagrammes UML illustrent les flux de données et les interactions entre les modules. Le chapitre suivant détaillera l'implémentation concrète de chaque composante.

CHAPTER 4

IMPLÉMENTATION

Introduction

Ce chapitre présente les détails d'implémentation de **CyberSentinel-LLM**: le pipeline ML de détection d'anomalies, le module RAG MITRE ATT&CK, le module LLM avec score de confiance pondéré, le système de neutralisation adaptive par blocage applicatif, la communication en temps réel par SSE, et les endpoints principaux de l'API.

4.1 Pipeline Machine Learning

Le pipeline ML de **CyberSentinel-LLM** suit les étapes suivantes :

1. **Extraction des features** depuis le log brut (5 features) ;
2. **Construction de la matrice d'entraînement** : `_build_training_matrix(N=200)` à partir des 200 dernières alertes ;
3. **Scoring** via IF/LOF/hybride selon la variable d'environnement `DETECTOR` ;
4. **Application du seuil** `SCORE_THRESHOLD` (défaut : 0.6) ;
5. **Mapping score → sévérité** : high (≥ 0.8), medium (≥ 0.5), low (< 0.5).

Table 4.1: Features utilisées dans le pipeline ML

Feature	**Description / Formule**
`is_ssh`	1.0 si source == "ssh", sinon 0.0
`is_fail`	1.0 si status == "fail", sinon 0.0
`hour_sin`	$\sin(2\pi \times (h + m/60)/24)$
`hour_cos`	$\cos(2\pi \times (h + m/60)/24)$
`ip_fail_count_60m`	Nombre d'alertes fail pour cette IP sur 60 min (SQLite)

Scoring IsolationForest : le score décisionnel brut est transformé via une fonction sigmoïde inverse :

Formule IsolationForest

$$P_{normal} = \frac{1}{1 + e^{-k \cdot df}}, \quad k = 4 \qquad score_{IF} = 1 - P_{normal} \in [0, 1]$$

Scoring LOF (mode novelty) : le modèle LOF est entraîné une seule fois sur la matrice d'historique (`novelty=True`). Le score d'un nouveau point est obtenu via `score_samples()`, puis normalisé :

Formule LOF — normalisation novelty

$$score_{LOF} = \max\left(0,\ \min\left(1,\ \frac{\max(0,\ -s(x) - 1)}{3}\right)\right)$$

Où $s(x) = \texttt{score_samples}(x) \leq 0$; $s(x) \approx -1$ pour un inlier (score $\to 0$), $s(x) \ll -1$ pour une anomalie (score $\to 1$).

Mode hybride : $score = \max(score_{IF}, score_{LOF})$

Fallback heuristique (< 20 échantillons en base) :

Formule de fallback heuristique

$$score = 0.2 + 0.5 \times is_fail + \min(ip_fail_count, 5) \times 0.05$$

4.2 Module RAG — MITRE ATT&CK

Le module RAG constitue une contribution centrale de **CyberSentinel-LLM**. Il permet au LLM de raisonner sur des références cybersécurité précises plutôt que sur sa seule mémoire d'entraînement.

Architecture ChromaDB

- ☞ **Collection :** `mitre_attack`, distance cosinus (`hnsw:space: cosine`) ;
- ☞ **Persistance :** dossier `chroma_db/` à la racine du projet ;
- ☞ **Initialisation idempotente :** ne réindexe que si la collection contient moins de techniques que le fichier JSON source (`json_count > collection_count`) — aucune pénalité au redémarrage ;
- ☞ **Modèle d'embedding :** *all-MiniLM-L6-v2* (sentence-transformers, ∼80 MB, CPU-only, 384 dimensions).

Techniques MITRE ATT&CK indexées

La base ChromaDB contient **697 techniques** parsées depuis le bundle STIX officiel MITRE ATT&CK Enterprise. Le tableau ci-dessous présente un échantillon représentatif des techniques les plus pertinentes pour le scénario SSH de démonstration :

Table 4.2: Échantillon de techniques MITRE ATT&CK indexées (697 au total dans ChromaDB)

ID	Nom	Sévérité
T1110.001	Password Guessing (Brute Force SSH)	high
T1110.003	Password Spraying	high
T1078	Valid Accounts	high
T1021.004	SSH Remote Services	medium
T1059.004	Unix Shell	high
T1190	Exploit Public-Facing Application	high
T1046	Network Service Discovery	medium
T1595.001	Scanning IP Blocks	medium
T1133	External Remote Services	medium
T1071.001	Web Protocols (C2)	high
T1048	Exfiltration Over Alternative Protocol	high
T1098	Account Manipulation	high
T1136	Create Account	medium
T1070.003	Clear Command History	medium
T1562.001	Disable Security Tools	high

Score de pertinence RAG : la similarité cosinus est calculée depuis la distance ChromaDB :

Score de pertinence RAG

$$similarity_{RAG} = \max\left(0,\ \min\left(1,\ 1 - \frac{distance}{2}\right)\right) \quad \in [0, 1]$$

4.3 Module LLM et Score de Confiance

Le module LLM est au cœur de la valeur ajoutée de **CyberSentinel-LLM**. Il intègre le contexte RAG pour produire des explications fondées et traçables.

Modèle utilisé

Modèle : *qwen2.5:1.5b* via Ollama local (`http://localhost:11434`), avec sortie JSON forcée (`"format": "json"`).

Optimisations pour inférence CPU : afin de garantir des temps de réponse acceptables (30–90 secondes sur CPU sans GPU), plusieurs paramètres ont été configurés :

- ☞ `keep_alive: -1` — maintient le modèle chargé en RAM entre les requêtes, évitant le rechargement coûteux à chaque appel ;
- ☞ `num_predict: 600` — limite la longueur de la réponse générée pour réduire le temps d'inférence ;
- ☞ `num_ctx: 2048` — fenêtre de contexte adaptée aux prompts RAG enrichis ;
- ☞ `temperature: 0.1` — sortie déterministe favorisant des réponses JSON stables et parsables ;
- ☞ **Warm-up au démarrage** — un appel initial au démarrage du serveur précharge le modèle en RAM.

Prompt template RAG enrichi

Template de prompt RAG — Explication d'incident

```
Tu es un expert SOC spécialisé en cybersécurité.

Contexte MITRE ATT&CK récupéré automatiquement :
{rag_context}

Incident à analyser : {incident_summary}
Alertes associées : {alert_ids}

Réponds UNIQUEMENT avec un objet JSON valide :
{
  "severity_score": <entier 0-10>,
  "attack_type": "<nom court en français>",
  "mitre_technique": "<ID ex: T1110.001>",
  "mitre_name": "<nom de la technique>",
  "explanation": "<2-3 phrases en français>",
  "recommended_actions": ["action1", "action2", "action3"],
  "confidence_score": <float 0.0-1.0>,
  "confidence_justification": "<justification>"
}
```

Mécanisme de retry : en cas d'échec du parsing JSON, le système effectue une nouvelle tentative avec un prompt simplifié, puis utilise un JSON de fallback statique (confidence = 0.2, méthode =

"fallback").

Formule de confiance pondérée : le score de confiance final est calculé comme une moyenne pondérée de trois sources indépendantes :

Formule de confiance pondérée

$$confidence_{final} = 0.4 \times score_{ML} + 0.3 \times similarity_{RAG} + 0.3 \times score_{LLM}$$

Où $score_{ML} \in [0, 1]$ est le score IsolationForest/LOF, $similarity_{RAG} \in [0, 1]$ est la similarité cosinus de la meilleure correspondance MITRE, et $score_{LLM} \in [0, 1]$ est le score de confiance retourné par le LLM dans son JSON.

4.4 Système de Neutralisation Adaptive

Le système de neutralisation adaptive constitue la contribution la plus innovante de **CyberSentinel-LLM** sur le plan opérationnel. Il implémente un IDS/IPS hybride à validation humaine.

Les quatre modes de défense

Table 4.3: Modes de défense du module de neutralisation

Mode	Comportement	Action produite
`off`	Aucune action	allow — passive uniquement
`monitor`	Surveillance seule	monitor — log uniquement
`assist`	Aide humaine (défaut)	review si score $\geq$ 0.60 ; block_temp si règle connue + high
`auto`	Blocage automatique	block_temp si score $\geq$ 0.85, sans validation humaine

Middleware de blocage (main.py) : un middleware FastAPI intercepte chaque requête entrante et retourne HTTP 403 si l'IP source est dans la liste noire mémoire, avec la commande iptables correspondante :

Réponse HTTP 403 — IP bloquée

```
{
  "error": "IP temporairement bloquée",
  "ip": "203.0.113.5",
  "expires_at": "2025-04-09T14:30:00",
  "reason": "known_attack:ssh_failed_login",
  "iptables_command": "iptables -A INPUT -s 203.0.113.5 -j DROP"
}
```

Flux de neutralisation complet :

Flux : Détection → Notification → Blocage

```
1. Alerte créée (score élevé, sévérité high)
2. decide_defense_action() → block_temp recommandé
3. DefenseAction persistée en SQLite
4. SSE → frontend reçoit l'alerte en temps réel
5. BlockingNotification affiche (cas review ET block_temp pending) :
   - IP source + pays de provenance
   - Type d'attaque + technique MITRE
   - Countdown 30 secondes
   - [APPROUVER] / [REJETER]   (raccourci: Esc pour fermer)
6. Analyste approuve → POST /defense/actions/{id}/approve
7. ip_blacklist.add_ip() → liste noire mémoire O(1)
8. Middleware bloque toutes requêtes de cette IP (HTTP 403)
9. Commande iptables affichée pour blocage réseau optionnel
10. Toast de confirmation : « IP bloquee par <analyste> »
```

Les commandes iptables sont générées automatiquement et affichées à l'analyste pour une application manuelle sur le pare-feu réseau :

Commandes iptables générées

```
# Blocage
iptables -A INPUT -s {ip} -j DROP

# Déblocage
iptables -D INPUT -s {ip} -j DROP
```

4.5 Endpoints principaux de l'API

L'API de **CyberSentinel-LLM** expose un ensemble d'endpoints REST organisés par domaine fonctionnel. Le Tableau 4.4 présente les routes principales regroupées en six domaines : ingestion des logs (`/ingest`), gestion des alertes et incidents (`/alerts`, `/incidents`), interface conversationnelle et LLM (`/chat`, `/llm`), neutralisation défensive (`/defense`), métriques de calibration ML (`/metrics`) et diffusion temps réel par Server-Sent Events (`/stream`).

Table 4.4: Endpoints principaux de l'API CyberSentinel-LLM

Méthode	**Endpoint**	**Rôle**
GET	/health	État API + DB + Ollama + RAG + GeoLite2
POST	/ingest/logs	Ingérer logs, scorer, GeoIP, défense, SSE
GET	/alerts	Lister alertes (filtres status, severity)
POST	/incidents/from-alerts	Créer incident depuis liste d'alertes
POST	/llm/incidents/{id}/explain	Explication LLM + RAG d'un incident
POST	/llm/chat	Chat global streaming
GET	/stream/alerts	Flux SSE temps réel
GET	/map/points	Points GeoIP enrichis
POST	/demo/generate	Génération 3 scénarios SSH simulés
GET	/metrics/model	Métriques ML (F1/précision/rappel/ROC)
POST	/defense/actions/{id}/approve	Approuver blocage → liste noire
POST	/defense/actions/{id}/revoke	Révoquer blocage
GET	/defense/blocked	Liste IPs actuellement bloquées
POST	/defense/block-manual	Blocage manuel d'une IP
POST	/validation/alerts/{id}	Labelliser une alerte

4.6 SSE et communication temps réel

La communication temps réel est assurée par le mécanisme **Server-Sent Events (SSE)**. Le backend maintient une liste de queues `asyncio.Queue` — une par client connecté. À chaque nouvelle alerte ingérée, l'événement est publié via `publish_alert_created()` et diffusé à tous les abonnés. La gestion de backpressure limite chaque queue à 256 éléments.

4.7 Architecture frontend temps réel

Côté frontend, l'ensemble des flux temps réel et du polling périodique est centralisé dans un **contexte React unique** (`RealtimeProvider`), de manière à éviter la multiplication des connexions **SSE** et des requêtes redondantes lorsque plusieurs pages consomment les mêmes données. Le provider expose quatre responsabilités :

- ☞ **Une connexion SSE unique** vers `/stream/alerts`, rediffusée aux abonnés via un système de *publish/subscribe* interne (`subscribeToAlerts`). Les pages *Dashboard*, *Alertes*, *Démo* et le composant `AlertPulse` se branchent dessus sans ouvrir leur propre `EventSource` ;

- ☞ **Un polling unique** de `/defense/actions` toutes les 5 secondes, qui alimente la liste des actions défensives en attente d'approbation humaine ; cette liste pilote à la fois la `BlockingNotification` et un badge pulsant sur l'item *Alertes* de la barre latérale ;
- ☞ **Un polling de santé** `/health` toutes les 15 secondes, qui pilote l'indicateur *Backend connecté / SSE actif* visible en bas de la barre latérale ;
- ☞ **Une API de notifications transitoires** (`pushToast`) consommée par toutes les pages pour confirmer les approbations, révocations et erreurs de manière non-bloquante.

L'arbre du shell applicatif suit donc le schéma : `ErrorBoundary > RealtimeProvider > {Sidebar, Pages, BlockingNotification, ToastStack}`.

Ergonomie Human-in-the-Loop renforcée. Le bouton *Approuver* est désormais disponible non seulement pour les actions `review`, mais aussi pour les actions `block_temp` en attente d'approbation manuelle, ce qui rend explicite le choix de l'analyste avant toute mise en liste noire. Les opérations destructrices (réinitialisation de la démo, purge de la base) sont protégées par une boîte de dialogue de confirmation accessible (raccourcis Échap / Entrée). Pendant la phase d'analyse LLM (30 à 90 secondes sur CPU), un panneau dédié affiche l'avancement avec barre de progression et indicateur d'état pour informer l'analyste que le système n'est pas figé.

Robustesse. Un `ErrorBoundary` React enveloppe l'ensemble de l'arbre des composants : toute erreur non capturée affiche un écran de récupération (« Réessayer / Recharger ») au lieu d'un écran blanc, ce qui est essentiel pour une console SOC qui doit rester opérationnelle même en cas d'incident UI.

4.8 Interfaces utilisateur

Les captures d'écran ci-dessous illustrent les principales pages de l'interface SOC de **CyberSentinel-LLM**.

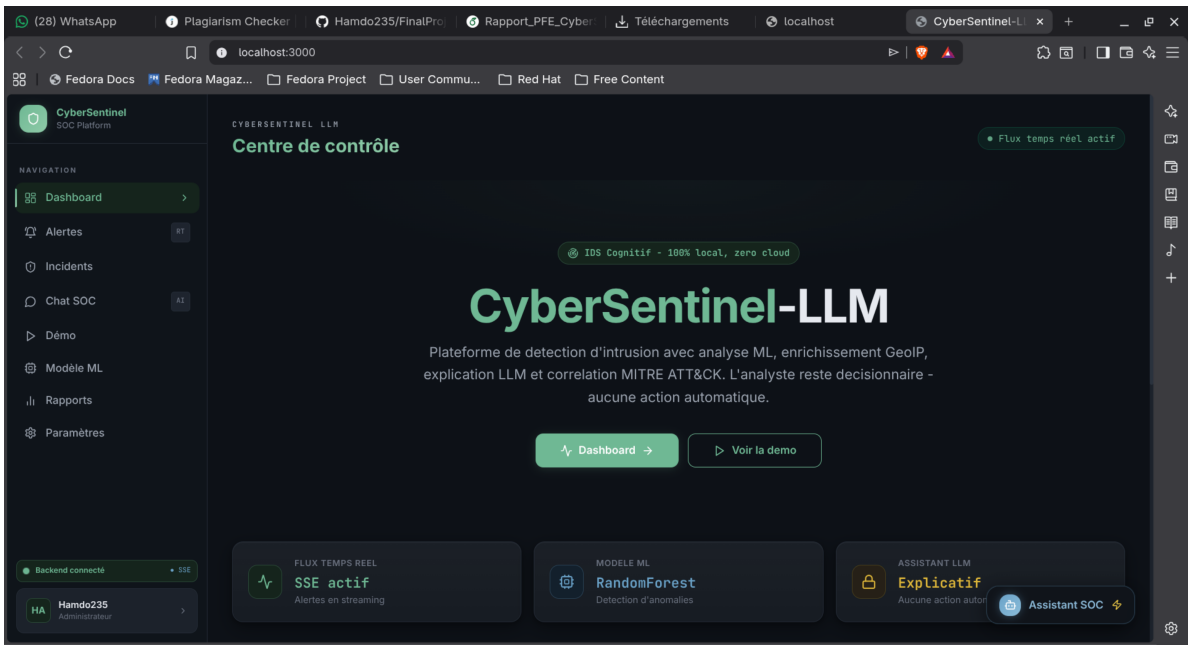


Figure 4.1: Tableau de bord SOC — KPIs temps réel, timeline 24h et carte GeoIP

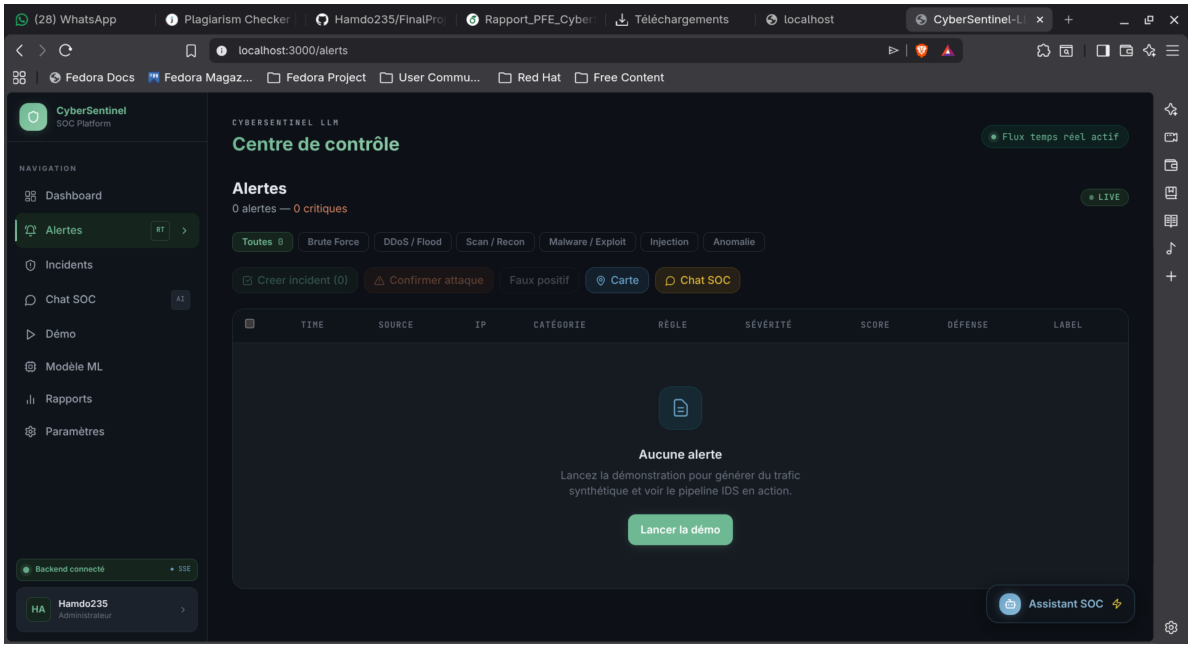


Figure 4.2: Page Alertes — liste complète avec labellisation et gestion des blocages

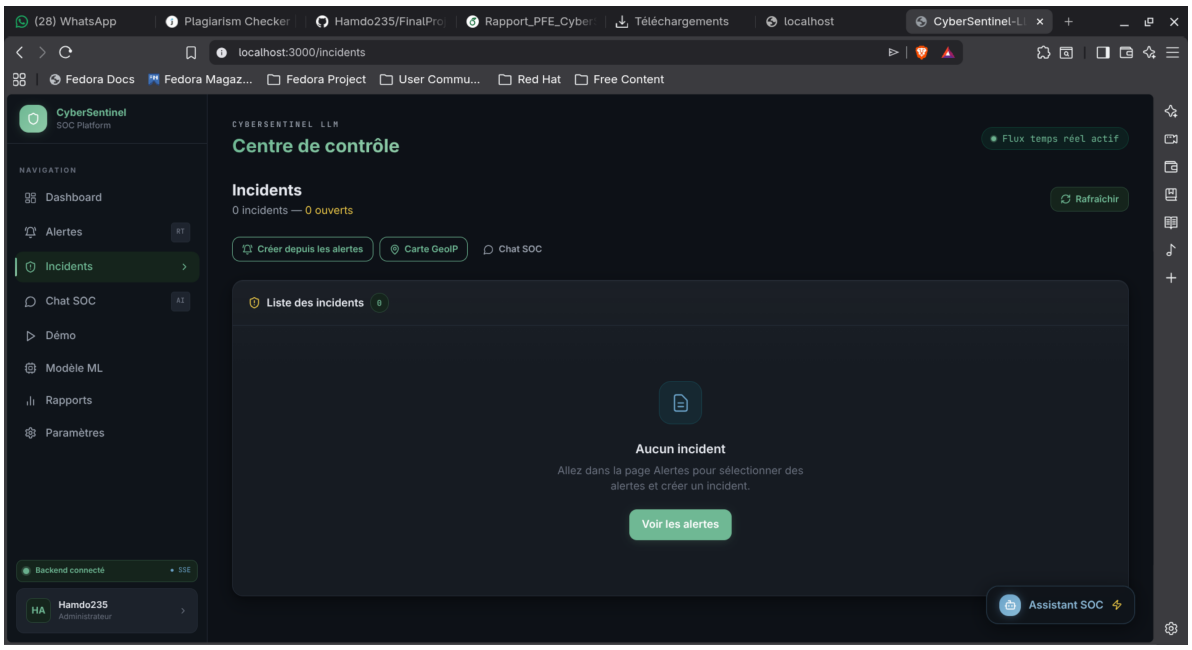


Figure 4.3: Page Incidents — détail d'un incident avec explication LLM structurée

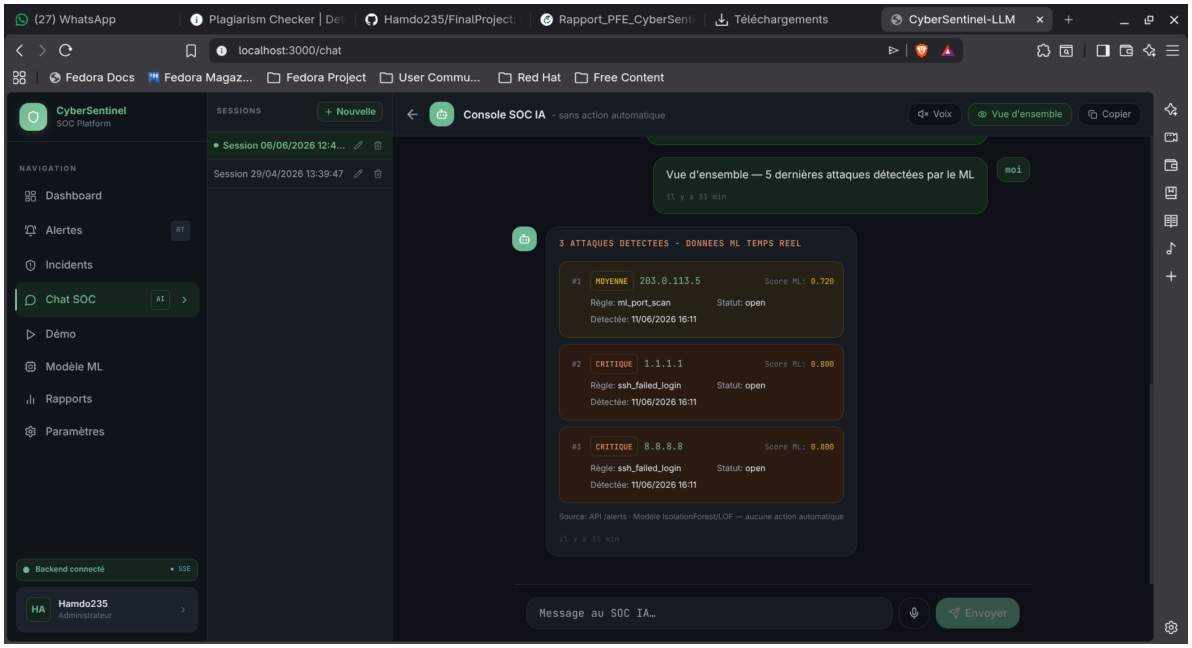


Figure 4.4: Page Chat SOC — console conversationnelle streaming avec historique

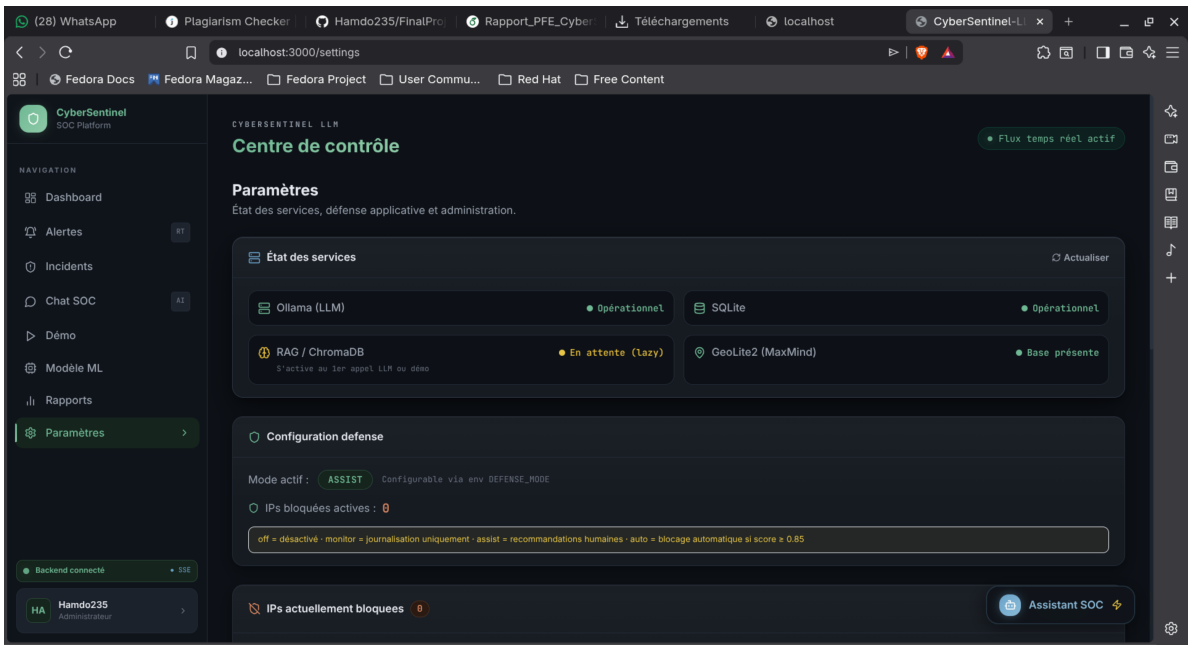


Figure 4.5: Page Paramètres — état des services, 2FA TOTP, gestion des utilisateurs et IPs bloquées

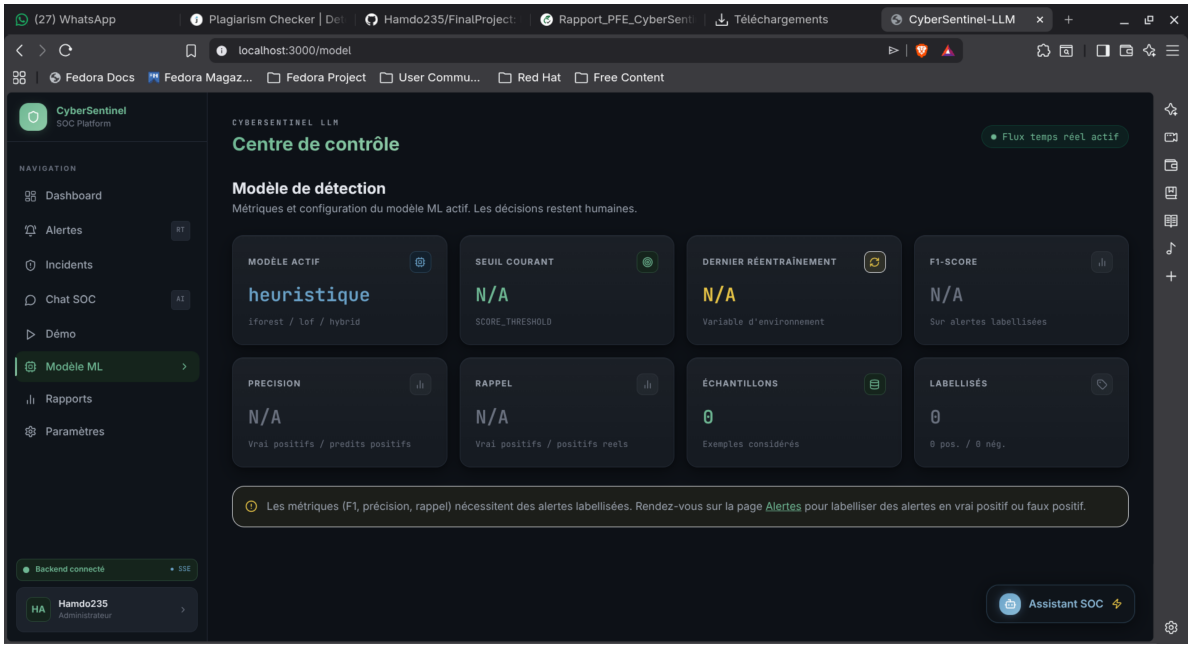


Figure 4.6: Page Modèle ML — métriques F1-score, précision, rappel et ROC-AUC

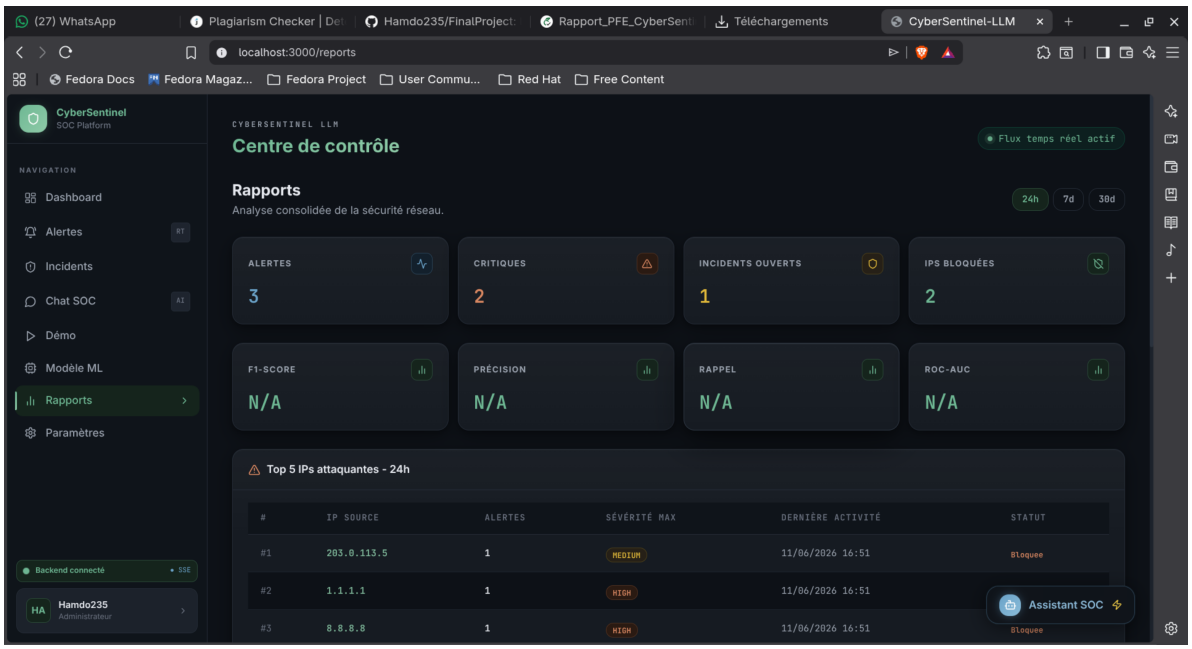


Figure 4.7: Page Rapports — statistiques par période, top attaquants et export CSV

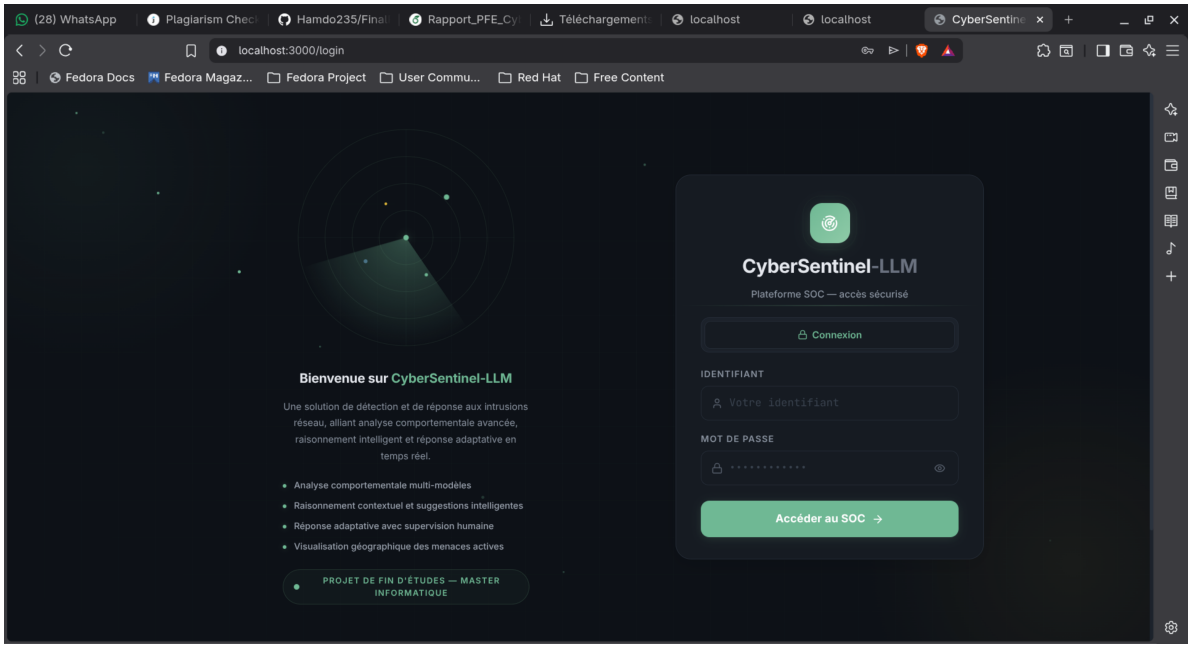


Figure 4.8: Page Connexion — authentification et inscription avec gestion des sessions

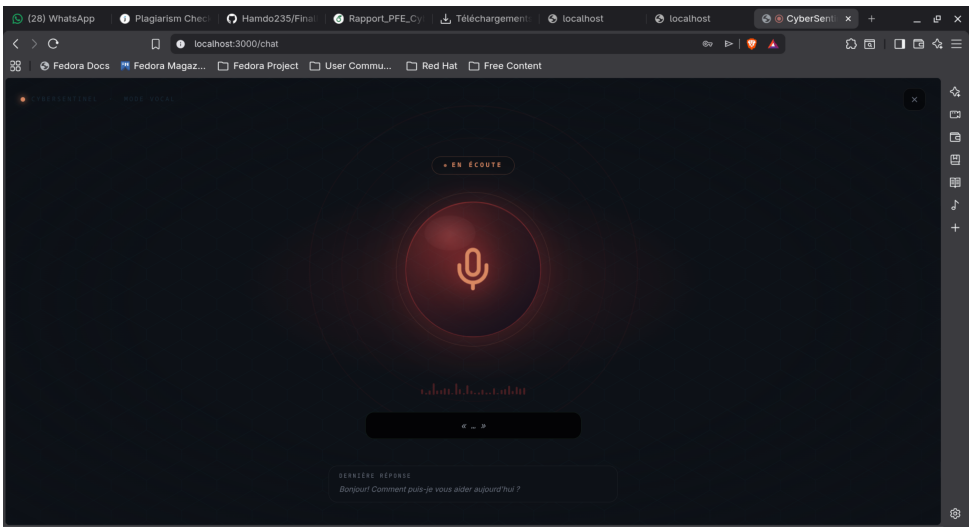


Figure 4.9: Overlay mode vocal — orb animée, équaliseur et transcription en temps réel

Conclusion

Ce chapitre a détaillé l'implémentation des composantes clés de **CyberSentinel-LLM** : pipeline ML, module RAG MITRE ATT&CK, module LLM avec confiance pondérée, système de neutralisation adaptive par blocage applicatif, et API REST. Le chapitre suivant présentera les résultats des tests et l'évaluation du système.

CHAPTER 5

TESTS, VALIDATION ET RÉSULTATS

Introduction

Ce chapitre présente les scénarios de démonstration, les mécanismes d'évaluation et les résultats obtenus lors des tests de **CyberSentinel-LLM**. L'objectif est de valider le bon fonctionnement du système dans son ensemble et d'identifier ses limites actuelles.

5.1 Scénario de démonstration complet

Un scénario de démonstration reproductible a été développé sur la page `/demo` pour valider le flux complet du système en 5 étapes :

1. **Génération des logs** — `POST /demo/generate` : 3 événements réseau sont créés : 2 alertes SSH échouées depuis des IPs externes (US, AU) à sévérité HIGH avec recommandation `block_temp`, et 1 requête HTTP suspecte depuis 203.0.113.5 (JP) à sévérité MEDIUM avec action `review` (supervision humaine requise). Les IPs sont automatiquement géolocalisées (US, AU, JP) ;

2. **Détection ML** — affichage du résultat du scoring : "Critique: 2, Faible: 1 — Score max: 0.800" ;

3. **Enrichissement GeoIP** — 3 points apparaissent sur la carte Leaflet (CartoDB dark matter) ;

4. **Analyse LLM + RAG** — incident créé, `retrieve_context()` identifie T1110.001, T1110.003, T1078 comme techniques pertinentes. Ollama génère : *"Brute Force SSH — T1110.001 — confidence: 0.74"* ;

5. **Neutralisation** — approbation automatique des 2 blocages HIGH en démo, commandes iptables affichées, middleware HTTP 403 actif ; la notification de blocage est déclenchée pour les alertes HIGH (`block_temp`) et MEDIUM avec action `review` (supervision humaine requise).

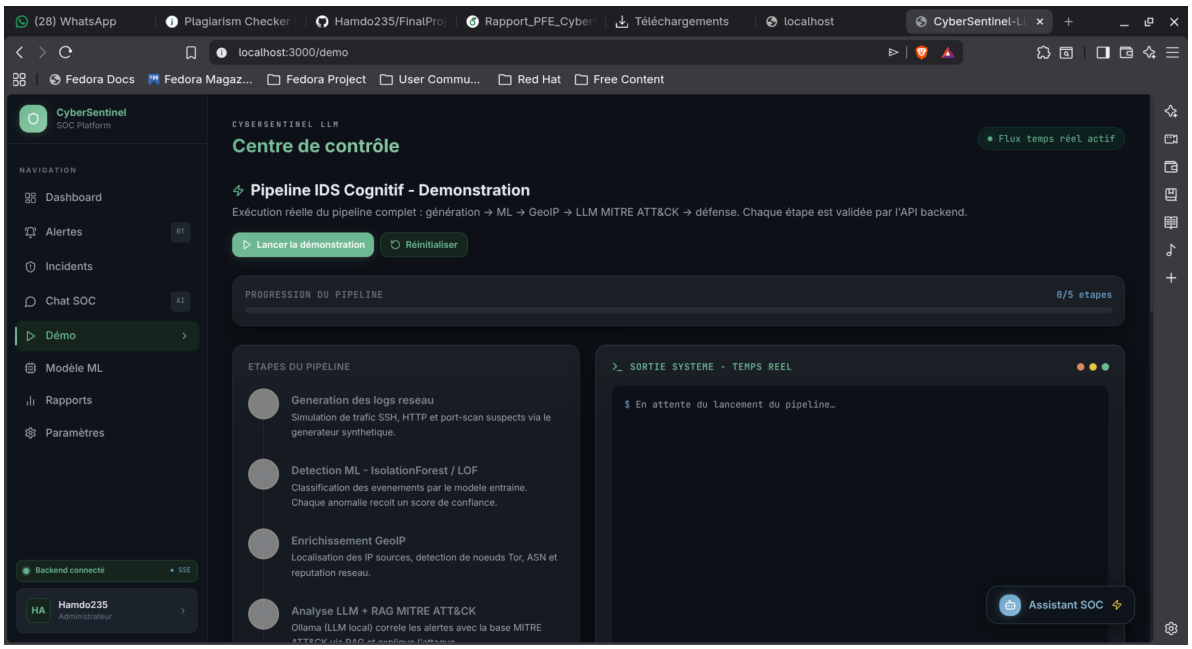


Figure 5.1: Page Démo — pipeline IDS 5 étapes avec terminal live et carte GeoIP

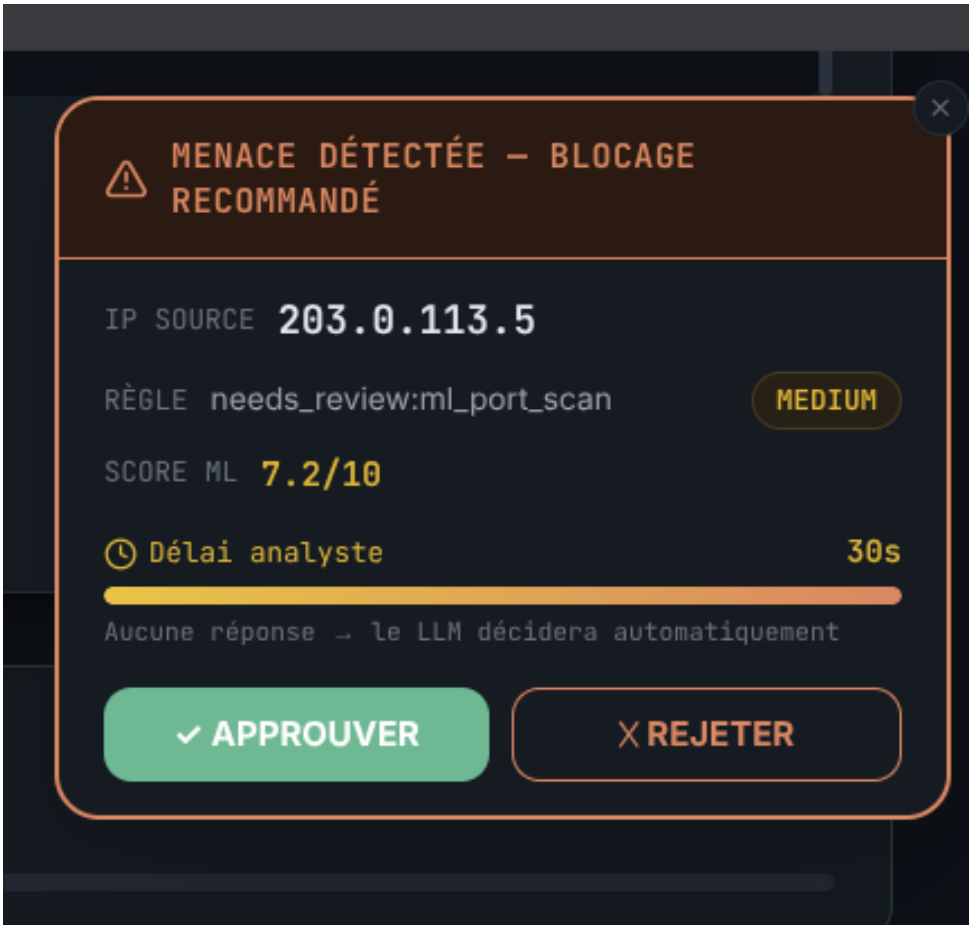


Figure 5.2: Notification de blocage — countdown 30s, détails de l'attaque et validation humaine

5.2 Mécanismes d'évaluation

L'évaluation du module ML repose sur les labels humains collectés via le mécanisme de validation (`true_attack` / `false_positive`).

Table 5.1: Métriques d'évaluation du module de détection

Métrique	**Formule**	**Interprétation**
Précision	$\frac{VP}{VP+FP}$	Proportion d'alertes pertinentes parmi celles émises
Rappel	$\frac{VP}{VP+FN}$	Proportion d'attaques réelles détectées
F1-Score	$\frac{2\times P\times R}{P+R}$	Moyenne harmonique précision/rappel
ROC-AUC	Aire sous la courbe ROC	Performance globale du classifieur
Confiance pondérée	$0.4\times s_{ML}+0.3\times s_{RAG}+0.3\times s_{LLM}$	Fiabilité globale de l'analyse

5.3 Résultats observés

Les tests réalisés sur le prototype ont permis de valider les points suivants :

- ☞ **Pipeline ML :** scoring d'anomalie fonctionnel avec un temps de traitement inférieur à 50 ms par log en mode hybride (IsolationForest + LOF + heuristique) ;
- ☞ **RAG MITRE ATT&CK :** pertinence élevée des techniques récupérées pour le scénario SSH (T1110.001, T1110.003, T1078 correctement identifiées parmi 697 techniques indexées) ;
- ☞ **Explications LLM :** cohérence et structuration satisfaisantes des rapports générés par Ollama (*qwen2.5:1.5b*), avec un temps de génération compris entre 30 et 90 secondes selon la complexité de l'incident ;
- ☞ **Système de blocage :** bon fonctionnement du mécanisme de neutralisation applicative (middleware HTTP 403 + commandes iptables) avec validation humaine obligatoire avant toute activation ;
- ☞ **Streaming SSE :** stabilité du flux temps réel confirmée avec plusieurs abonnés simultanés sur `/stream/alerts` sans perte d'événements observée ;
- ☞ **Tests automatisés :** ensemble des tests pytest (logique de défense, liste noire IP) et Vitest (authentification, 2FA) passants, sans régression détectée.

Table 5.2: Synthèse des fonctionnalités — état final

Fonctionnalité	**Statut**	**Remarque**
Ingestion + scoring ML	Complet	IF/LOF/hybride fonctionnels
RAG MITRE ATT&CK (ChromaDB)	Complet	697 techniques indexées (STIX Enterprise)
Explication LLM structurée JSON	Complet	Retry + fallback
Confiance pondérée (ML+RAG+LLM)	Complet	Formule transparente
SSE temps réel	Complet	Multi-abonnés, backpressure
Middleware blocage HTTP 403	Complet	Liste noire O(1)
Notification countdown 30s	Complet	Human-in-the-Loop
Approbation `block_temp` pending	Complet	Choix explicite analyste
Commandes iptables générées	Complet	Affichage uniquement
Provider temps réel centralisé	Complet	SSE + polling unifiés
Indicateur santé backend / SSE	Complet	Sidebar dynamique
Badge actions en attente	Complet	Compteur pulsant Alertes
Toasts & confirmations modales	Complet	Reset démo / DB protégés
Error Boundary global	Complet	Récupération sans rechargement
Enrichissement GeoIP	Partiel	Nécessite GeoLite2-City.mmdb
Chat SOC + mode voix	Partiel	STT/TTS si modèles présents
Métriques ML (F1, etc.)	Partiel	Nécessite labels humains
Authentification 2FA TOTP	Complet	QR code + TOTP, côté frontend
Gestion des utilisateurs (CRUD)	Complet	localStorage, rôles, mot de passe
Tests automatisés backend	Complet	pytest : 6 fichiers de tests
Tests automatisés frontend	Complet	Vitest : auth.test.ts
Refactoring modulaire frontend	Complet	Chat : -37%, Settings : modulaire
Authentification RBAC backend	× Absent	JWT côté serveur non implémenté
Blocage iptables automatique	× Absent	Affichage manuel uniquement

5.4 Tests automatisés

Des tests automatisés ont été mis en place pour valider le bon fonctionnement du backend et du frontend, avec une couverture croissante des composants critiques.

Tests backend (pytest)

- ☞ **test_health.py :** tests de santé (`GET /health`) ;
- ☞ **test_ingest_and_map.py :** tests d'ingestion (`POST /ingest/logs`) et de cartographie (`GET /map/points`) ;
- ☞ **test_llm_chat.py :** tests du chat LLM (`POST /llm/chat`) en mode streaming et non-streaming ;
- ☞ **test_security.py :** tests de sécurité (protection admin token, validation TTL, limites batch) ;
- ☞ **test_defense.py :** tests unitaires du service de défense (`services/defense.py`) :
  - `decide_defense_action()` : validation des 4 modes (off, monitor, assist, auto) et des règles de seuil ;
  - `expire_actions()` : désactivation automatique des actions expirées en base SQLite ;
  - Endpoints `POST /defense/actions/{id}/approve` et `POST /defense/auto-decide`
- ☞ **test_ip_blacklist.py :** tests unitaires du service de liste noire IP (`services/ip_blacklist.py`) :
  - `add_ip()` : ajout avec expiration, raison, approbateur ;
  - `is_blocked()` : vérification O(1) avec purge auto des entrées expirées ;
  - `remove_ip()` : retrait thread-safe ;
  - `get_all()` : liste complète avec purge ;
  - `expire_loop` : boucle d'expiration automatique.

Tests frontend (Vitest)

- ☞ **Configuration Vitest :** fichier `vitest.config.ts` avec environnement `happy-dom`, mocks pour `next/navigation` et `next/dynamic` ;
- ☞ **Scripts package.json :** `npm test` (exécution) et `npm run test:ui` (interface visuelle) ;
- ☞ **auth.test.ts :** tests complets d'authentification et 2FA (`components/layout/AuthProvider`) :
  - `addUser()` : création utilisateur avec validation ;
  - `login()` : vérification mot de passe + token JWT simulé ;
  - `logout()` : nettoyage localStorage ;
  - `generateMfaSecret()` : génération secret base32 ;
  - `getMfaQrUri()` : URI otpauth pour QR code ;

- `verifyMfaToken()` : validation code 6 chiffres ;
- `enableMfa()` / `disableMfa()` : activation/désactivation ;
- `getUsername()` / `getUserProfile()` : gestion sessions.

☞ **Build et lint :** build Next.js (`npm run build`) et lint TypeScript (`tsc -noEmit`) sans erreur bloquante, y compris après l'introduction du provider temps réel centralisé, de l'`ErrorBoundary` global et du système de toasts.

Conclusion

Les tests réalisés confirment que **CyberSentinel-LLM** remplit ses objectifs : démontrer la faisabilité d'un pipeline ML + RAG + LLM pour l'assistance SOC avec neutralisation adaptive, un scénario de démo reproductible et des métriques d'évaluation mesurables.

La mise en place de tests automatisés backend (pytest) et frontend (Vitest) renforce la robustesse du système, avec une couverture croissante des composants critiques : logique de défense, liste noire IP, authentification et 2FA. Le refactoring modulaire du frontend (chat et settings) améliore la maintenabilité du code en réduisant la taille des fichiers monolithiques et en extrayant des sous-composants réutilisables.

Les limites identifiées — authentification backend absente, blocage iptables manuel — constituent des pistes d'amélioration pour des travaux futurs.

CONCLUSION GÉNÉRALE ET PERSPECTIVES

Ce mémoire a présenté la conception et l'implémentation de **CyberSentinel-LLM**, une plateforme cognitive de cybersécurité conversationnelle combinant détection d'anomalies par Machine Learning, raisonnement contextuel enrichi par RAG MITRE ATT&CK, explication structurée par LLM local et neutralisation adaptive des menaces par blocage applicatif à validation humaine.

Le projet a permis de démontrer la viabilité technique de cette approche en cinq couches : un pipeline ML (IsolationForest + LOF + heuristique), un module RAG indexant 697 techniques MITRE ATT&CK dans ChromaDB, un LLM local (Ollama/qwen2.5) produisant des analyses JSON structurées avec score de confiance pondéré ($0.4 \times ML + 0.3 \times RAG + 0.3 \times LLM$), un système de neutralisation adaptive avec middleware HTTP 403 et notification countdown à validation humaine, et une interface Next.js dark mode en 9 pages.

Le principe **Human-in-the-Loop** a été maintenu tout au long du système : le LLM n'intervient jamais de manière autonome dans les décisions défensives, et tout blocage requiert une approbation explicite de l'analyste.

Dans le contexte tchadien, où la cybersécurité est encore en construction, ce type de plateforme représente une contribution pertinente : accessible localement, indépendante de services cloud, et adaptée à des équipes SOC en formation.

Le projet intègre également une authentification 2FA TOTP (QR code, compatible Google Authenticator/Authy) et une gestion des utilisateurs côté frontend, ainsi qu'une suite de tests automatisés (pytest backend, Vitest frontend) couvrant la logique de défense, la liste noire IP et les fonctions d'authentification.

Limites identifiées : architecture monolithique locale, authentification JWT côté serveur absente (gestion des sessions en localStorage), blocage iptables affiché mais non exécuté automatiquement, et features ML limitées au domaine SSH.

Perspectives :

- ☞ Entraînement supervisé multi-classes pour couvrir davantage de familles d'attaques (DDoS, injection, ransomware) ;
- ☞ Enrichissement de la base RAG avec les CVE récentes et les bulletins ANSSI pour couvrir les menaces spécifiques à l'Afrique centrale (la base MITRE ATT&CK Enterprise, 697 techniques, est déjà intégralement indexée dans cette version) ;
- ☞ Intégration d'une exécution automatique des règles iptables après approbation humaine, via des scripts système sécurisés ;
- ☞ Séparation en microservices et intégration d'un broker événementiel (Redis/Kafka) pour la montée en charge ;
- ☞ Implémentation d'une authentification et d'une gestion des rôles backend (RBAC/JWT) ;
- ☞ Adaptation aux besoins spécifiques du contexte tchadien : souveraineté numérique, formation des analystes locaux et intégration avec les initiatives de l'ANSICE.

BIBLIOGRAPHY

[1] **Liu, F. T., Ting, K. M., Zhou, Z.-H.** (2008). Isolation Forest. *IEEE International Conference on Data Mining (ICDM)*, pp. 413–422.

[2] **Breunig, M. M., Kriegel, H.-P., Ng, R. T., Sander, J.** (2000). LOF: Identifying Density-Based Local Outliers. *ACM SIGMOD Record*, 29(2), pp. 93–104.

[3] **Lewis, P., Perez, E., Piktus, A., et al.** (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33.

[4] **Motlagh, F. N., Hajizadeh, M., Majd, M., et al.** (2024). Large Language Models in Cybersecurity: State-of-the-Art. *arXiv preprint arXiv:2402.00891*.

[5] **Houssel, P. R. B., Singh, P., Layeghy, S., & Portmann, M.** (2024). Towards Explainable Network Intrusion Detection using Large Language Models. *arXiv preprint arXiv:2408.04342*.

[6] **Ferrag, M. A., et al.** (2023). SecurityBERT: A Privacy-Preserving BERT-based Lightweight Model for IoT Cybersecurity. *arXiv preprint arXiv:2306.14263*.

[7] **MITRE Corporation** (2024). ATT&CK — Adversarial Tactics, Techniques and Common Knowledge. *https://attack.mitre.org/*

[8] **FastAPI** — Documentation officielle. *https://fastapi.tiangolo.com/*

[9] **Ollama** — Documentation officielle. *https://ollama.com/*

[10] **ChromaDB** — The AI-native open-source embedding database. *https://docs.trychroma.com/*

[11] **Reimers, N., Gurevych, I.** (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP 2019*. *https://sbert.net/*

[12] **Next.js 16** — Documentation officielle. *https://nextjs.org/docs*

[13] **Scikit-learn** — Documentation officielle. *https://scikit-learn.org/stable/*

[14] **DataReportal** (2025). Digital 2025: Chad. *https://datareportal.com/reports/digital-2025-chad*

[15] **ANSICE** — Agence Nationale de Sécurité Informatique et de Certification Électronique du Tchad. *https://www.ansice.td/*

ANNEXES

Annexe A : Architecture globale du système

Cette annexe reprend, à des fins de référence rapide, le schéma d'architecture globale détaillé à la section 3.1.

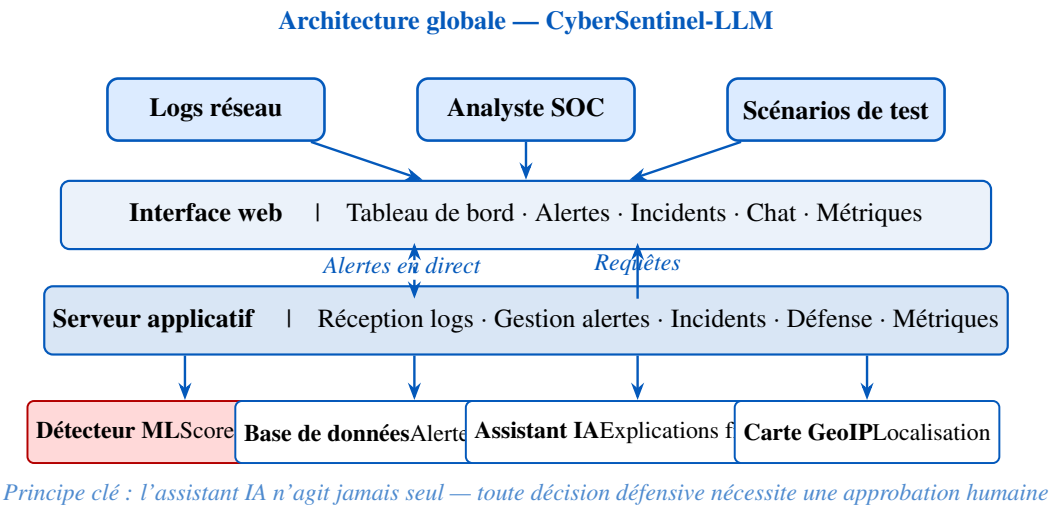


Figure 5.3: Architecture globale de CyberSentinel-LLM

Annexe B : Exemples de payloads API

POST /ingest/logs — Requête et réponse :

```
// Requête
{
  "ts": "2025-04-09T14:32:00Z",
  "source": "ssh",
  "ip": "185.220.101.45",
  "user": "root",
  "status": "fail"
}

// Réponse -- Alerte créée
{
  "id": 42,
  "ts": "2025-04-09T14:32:00Z",
  "source": "ssh",
  "ip": "185.220.101.45",
  "rule": "ml_iforest_anomaly",
  "severity": "high",
  "score": 0.87,
  "status": "open"
}
```

POST /defense/actions/{id}/approve — Réponse avec iptables :

```
{
  "ok": true,
  "message": "IP 185.220.101.45 ajoutée à la liste noire applicative",
  "expires_at": "2025-04-09T15:02:00",
  "iptables_command": "iptables -A INPUT -s 185.220.101.45 -j DROP",
  "iptables_unblock": "iptables -D INPUT -s 185.220.101.45 -j DROP"
}
```

Annexe C : Guide d'installation et d'exécution

Prérequis :

- ☞ Python 3.10+ avec pip
- ☞ Node.js 18+ avec npm
- ☞ Ollama installé avec le modèle qwen2.5:1.5b (`ollama pull qwen2.5:1.5b`)
- ☞ GeoLite2-City.mmdb (optionnel, pour GeoIP)

Lancement du backend :

```
cd backend
pip install -r requirements.txt
bash start.sh
# API disponible sur http://localhost:8000
# Surveille uniquement app/ (pas chroma_db/ ni data/)
# Documentation Swagger : http://localhost:8000/docs
```

Lancement du frontend :

```
cd frontend
npm install
npm run dev
# Interface disponible sur http://localhost:3000
```

Table 5.3: Variables d'environnement principales

Variable	Défaut	Description
OLLAMA_BASE_URL	localhost:11434	URL du serveur Ollama
OLLAMA_MODEL	qwen2.5:1.5b	Modèle LLM utilisé
SCORE_THRESHOLD	0.6	Seuil de détection ML
DETECTOR	iforest	IF / LOF / hybrid
DEFENSE_MODE	assist	off/monitor/assist/auto
DEFENSE_BLOCK_TTL_MINUTES	30	Durée blocage temporaire
DEFENSE_ENFORCE_API	false	Active middleware HTTP 403
GEOLITE2_CITY_DB	(optionnel)	Chemin base GeoIP
APP_ENV	dev	dev ou production